

College Code: 9209

N.S.N College of Engineering and Technology

PROJECT DOCUMENTATION

EVENTFORCE MANAGEMENT SYSTEM

Team Members

S.No	Name	Role
1	Sanjaykumar J	Team Lead
2	Ramachandran R	Team Member
3	Sanjay G	Team Member
4	Sarathi P	Team Member

ABSTRACT

Event planning involves client bookings, venue reservations, vendor coordination and feedback collection. When handled through spreadsheets and calls, it causes scattered data, missed reminders and venue double bookings. The **EventForce Management System** is a Salesforce solution that centralises these operations using six custom objects - Event, Client, Vendor, Venue, Feedback and the EventVendor junction object.

The system uses validation rules, formula fields, an approval process for cancellations and a record-triggered flow for 3-day client reminders. Apex classes, a trigger and a Batch/Schedulable job handle venue availability, double-booking prevention and nightly event completion. A Lightning App, reports and a dashboard give operational visibility, while profiles, roles, permission sets and sharing rules secure the data. The solution was built and tested in a Salesforce Developer Edition org.

Keywords: Salesforce, Event Management, Flow, Apex, Batch Apex, Lightning App, Dashboard, Sharing Rules.

TABLE OF CONTENTS

ABSTRACT	2
1. USER STORY.....	5
1.1 Background Scenario.....	5
1.2 Users of the System	5
1.3 User Stories	6
1.4 Acceptance Criteria	6
2. INTRODUCTION	7
2.1 Overview	7
2.2 Problem Statement.....	7
2.3 Objectives of the Project.....	8
2.4 Scope of the Project.....	8
2.5 Why Salesforce?.....	9
2.6 Technology Stack	9
2.7 Expected Benefits	10
2.8 Organisation of the Report.....	10
3. PROJECT OVERVIEW	11
3.1 Phase 1: Requirement Analysis & Planning.....	12
3.2 Phase 2: Backend Development & Configurations.....	15
3.2.1 Milestone 1: Salesforce Account.....	15
3.2.2 Milestone 2: Custom Objects	16
3.2.3 Milestone 3: Tabs.....	17
3.2.4 Milestone 4: Fields and Relationships.....	18
3.2.5 Milestone 5: Validation Rules	22
3.2.6 Milestone 6: Approval Process	23
3.2.7 Milestone 7: Flows	25
3.2.8 Milestone 8 and 9: Apex Classes and Triggers	27
3.2.9 Milestone 10: Asynchronous Apex	32
3.3 Phase 3: UI/UX Development & Customization	36
3.3.1 Milestone 11: Lightning App.....	36
3.3.2 Milestone 12: Reports	38
3.3.3 Milestone 13: Dashboards.....	40
3.4 Phase 4: Data Migration, Testing & Security	42
3.4.1 Data Migration.....	42
3.4.2 Milestone 14: Profiles	44
3.4.3 Milestone 15: Roles	45
3.4.4 Milestone 16: Users	46

3.4.5 Milestone 17: Permission Sets.....	47
3.4.6 Milestone 18: Sharing Settings	48
3.4.7 Testing.....	49
3.5 Phase 5: Deployment, Documentation & Maintenance.....	52
3.5.1 Deployment Scope and Limitations	52
3.5.2 System Documentation.....	53
3.5.3 Maintenance and Monitoring.....	53
4. CONCLUSION.....	56
4.1 Objectives Achieved.....	56
4.2 Learning Outcomes.....	56
4.3 Limitations.....	57
4.4 Future Enhancements.....	57
4.5 References	57
APPENDIX A: DATA DICTIONARY	58
APPENDIX B: GLOSSARY	60

1.USER STORY

1.1 Background Scenario

Meera is an event coordinator who tracks bookings in Excel, venues in a calendar and vendors in chat groups. Last month two events were booked in the same hall on the same date because one list was not updated. The owner, Rajesh, also had no single place to see upcoming events, vendor workload or client feedback. EventForce gives Meera one workspace with automatic conflict checks and reminders, gives Rajesh a live dashboard, and gives clients and vendors limited, secure access.



Figure 1: Existing manual process versus EventForce process

1.2 Users of the System

Role	Description	Main Goal
Client	Individual or company that books an event	Book events, track status and give feedback
Event Coordinator	Staff member who plans and runs events	Plan events without conflicts and keep clients informed
Vendor	External service provider (catering, decoration, sound, photography)	Know which events they are assigned to
Venue Manager	Person responsible for halls and venues	Keep venue status and availability accurate
Event Administrator	Senior manager / system owner	Approve cancellations, monitor operations, manage security

1.3 User Stories

ID	User Story	Delivered By
US-01	As a Client, I want to book an event with my preferred date and venue so that I need not call repeatedly.	Event__c, Client__c
US-02	As a Coordinator, I want double bookings blocked so that venue conflicts never happen.	PreventDoubleBooking trigger
US-03	As a Coordinator, I want the budget calculated automatically so that I can quote quickly.	Event_Budget__c formula
US-04	As a Coordinator, I want many vendors per event and many events per vendor so that all services are tracked.	EventVendor__c junction
US-05	As a Client, I want a reminder 3 days before my event.	Record-Triggered Flow
US-06	As an Administrator, I want cancellations approved by me so that none happen by mistake.	Approval Process
US-07	As a Venue Manager, I want venue status updated automatically.	VenueStatusHelper
US-08	As an Administrator, I want past events marked Completed automatically.	BatchCompleteEvents
US-09	As a Vendor Manager, I want to see only relevant events so that client data stays protected.	Sharing rules, OWD Private
US-10	As a Manager, I want a dashboard of events, vendors and ratings.	Reports and Dashboard

1.4 Acceptance Criteria

- A venue cannot be assigned to two active events on the same date.
- An invalid email address cannot be saved on a Client or Vendor record.
- A cancellation request is routed to the Event Administrator for approval.
- Past events are changed to Completed by the nightly job.
- Each user sees only the data permitted for their role.

2.INTRODUCTION

1.5 Overview

Event planning is a coordination-heavy industry. A single wedding or corporate conference may involve a client, a venue, five to ten vendors, a team of coordinators and a fixed date that cannot be moved. A small mistake in communication can damage the reputation of the whole company. Digital tools that bring data, people and processes together are therefore essential for modern event companies.

Salesforce is the world's leading cloud-based Customer Relationship Management (CRM) platform. Beyond CRM, it is also a platform-as-a-service (PaaS) that allows organisations to build their own custom applications using clicks (declarative tools) and code (Apex, Lightning Web Components). The EventForce Management System uses this platform to deliver a complete event management application without maintaining any servers or infrastructure.

1.6 Problem Statement

Event planning companies manage client bookings, vendor coordination, venue reservations and post-event feedback at the same time. In many organisations these activities are still handled through spreadsheets, emails, phone calls and messaging groups. Because client details, vendor assignments, venue calendars and feedback are stored in separate places, the coordinators have no single, reliable view of the event pipeline.

This lack of a centralised system leads to specific operational problems: the same venue can be booked for two events on the same date, reminders to clients depend on manual follow-up, event cancellations are made without management approval, completed events are not updated on time, and every employee can see confidential client and vendor data. Management also cannot measure vendor workload, venue usage or client satisfaction without collecting data by hand.

Therefore, the problem addressed by this project is: to design and implement a Salesforce-based EventForce Management System that centralises Events, Clients, Vendors, Venues and Feedback in one platform; automates reminders, cancellation approvals, venue availability, double-booking prevention and event completion; and secures the data through role-based access, while providing reports and dashboards for operational visibility.

Key challenges identified

Challenge in the Manual Process	Impact on the Business	EventForce Solution
Client, vendor, venue and feedback data stored in separate files	No single view; duplicate and inconsistent data	Six connected custom objects in one Salesforce org
Venues booked twice for the same date	Last-minute changes, extra cost, unhappy clients	PreventDoubleBooking trigger and VenueStatusHelper class
Vendors linked to events through phone and chat	Unclear assignments and workload	EventVendor junction object (many-to-many)
Reminders sent manually	Missed reminders and last-minute confusion	Record-Triggered Flow - client reminder 3 days before the event
Cancellations done without management knowledge	Unplanned losses and conflicts with vendors	Cancellation approval process with email alerts
Past events not updated	Wrong reports and venues shown as booked	BatchCompleteEvents - nightly Batch and Schedulable Apex
Invalid email addresses and manual budget calculation	Undelivered emails and quoting errors	Email_Valid_Address validation rule and Event_Budget_c formula
All staff can see all records	Risk to confidential client and vendor data	Profiles, roles, permission sets, OWD Private and sharing rules
No management reporting	Slow and uninformed decisions	Reports and EventForce Operations Dashboard

1.7 Objectives of the Project

1. To build a centralised Salesforce application that manages events, clients, vendors, venues and feedback.
2. To automate routine activities such as reminders, venue status updates and event completion.
3. To enforce data quality using validation rules and formula fields.
4. To introduce a controlled approval process for event cancellations.
5. To prevent venue double bookings using Apex triggers.
6. To provide role-based access using profiles, roles, permission sets and sharing rules.
7. To provide management with reports and dashboards for decision making.

1.8 Scope of the Project

The project covers the complete lifecycle of an event inside the Salesforce platform - from booking to feedback. The following items are in scope: custom object design, fields and relationships, validation rules, one approval process, one record-triggered flow, two Apex

classes and one trigger, a Batch and Schedulable Apex job, a Lightning App, reports and a dashboard, data migration through the Data Import Wizard, and the complete security model.

The following items are out of scope for this academic prototype: live production deployment, sandbox-to-production migration, payment gateway integration, and external client portals. These are discussed as future enhancements in the conclusion.

1.9 Why Salesforce?

Requirement	How Salesforce Meets It
Rapid development	Declarative tools (Object Manager, Flow Builder, Report Builder) allow features to be delivered in hours instead of weeks.
Automation	Flow, Approval Processes, Apex Triggers and Scheduled Apex cover simple to complex business logic.
Security	Profiles, roles, permission sets and sharing rules give field-level and record-level control.
Analytics	Built-in reports and dashboards need no external BI tool.
Scalability	Multi-tenant cloud architecture scales automatically with data and users.
User experience	Lightning Experience provides a modern, responsive interface that works on desktop and mobile.

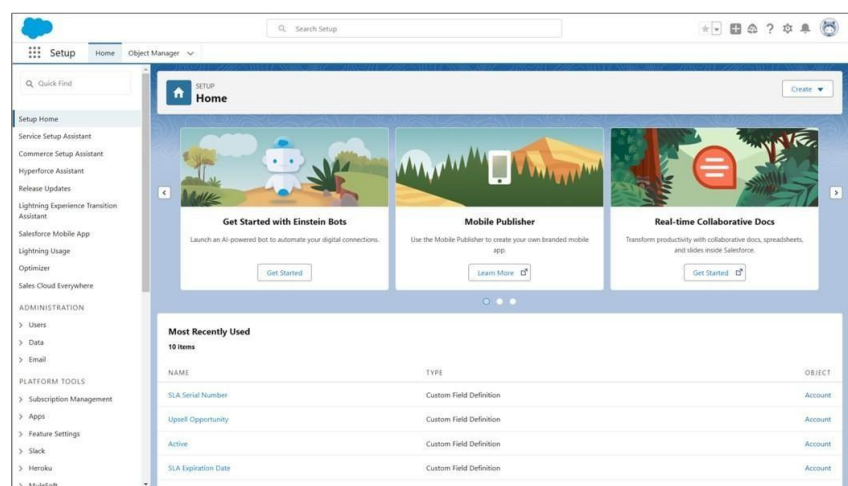


Figure 2: Salesforce Developer Edition - Home page of the org

1.10 Technology Stack

Layer	Technology Used
Platform	Salesforce Developer Edition (Lightning Experience)
Data model	Custom objects, lookup and master-detail relationships, junction object

Layer	Technology Used
Business logic (declarative)	Validation rules, formula fields, approval process, record-triggered flow
Business logic (code)	Apex classes, Apex triggers, Batch Apex, Schedulable Apex
User interface	Lightning App Builder, custom tabs, list views
Analytics	Reports and Dashboards
Security	Profiles, roles, users, permission sets, OWD and sharing rules
Data loading	Data Import Wizard (CSV files)

1.11 Expected Benefits

- A single source of truth for all event-related data.
- Zero venue conflicts because the system blocks them at save time.
- Less manual work because reminders, status updates and completion are automatic.
- Faster management decisions due to live dashboards.
- Better client experience through timely communication and feedback collection.
- Strong data protection because each user sees only what they need.

1.12 Organisation of the Report

Chapter 1 presents the user story and requirements. Chapter 2 introduces the project. Chapter 3 describes the project in five phases: Requirement Analysis and Planning, Backend Development and Configuration, UI/UX Development, Data Migration, Testing and Security, and Deployment, Documentation and Maintenance. Chapter 4 gives the conclusion and future scope. The appendices contain the data dictionary and a glossary of terms.

2. PROJECT OVERVIEW

The EventForce Management System was delivered in five phases. Each phase has a clear objective and a set of milestones. The table below gives the complete roadmap, and the sections that follow describe every phase and milestone in detail with the steps performed and the screenshots captured.

Phase	Title	Milestones
1	Requirement Analysis & Planning	Scope, stakeholder mapping, functional scope, roadmap
2	Backend Development & Configurations	Milestones 1 - 10 (account, objects, tabs, fields, validation, approval, flow, Apex, trigger, batch)
3	UI/UX Development & Customization	Milestones 11 - 13 (Lightning App, reports, dashboards)
4	Data Migration, Testing & Security	Data import, Milestones 14 - 18 (profiles, roles, users, permission sets, sharing), testing
5	Deployment, Documentation & Maintenance	Deployment scope, documentation, monitoring and maintenance

2.1 Phase 1: Requirement Analysis & Planning

2.1.1 Project Scope and Objective

The objective of the project is to build the EventForce Management System in Salesforce so that event planning operations such as client bookings, vendor coordination, venue reservations and feedback collection are centralised in one platform. During the analysis stage, the problems of the manual process (Chapter 2) were converted into measurable goals: no double bookings, automatic reminders, controlled cancellations, role-based visibility and management dashboards.

The scope was kept realistic for a Developer Edition org. Features were chosen so that every major area of the Salesforce platform - data model, declarative automation, Apex, UI, analytics and security - is demonstrated at least once, while the solution remains simple enough to be tested completely.

2.1.2 Stakeholder Mapping

Stakeholders were identified and mapped to roles that later became Salesforce profiles and roles. The mapping makes sure that every person using the system has the minimum access required for their work.

Stakeholder	Responsibilities	Objects Used	Planned Access Level
Client	Request bookings, view own events, submit feedback	Event, Client, Feedback	Read own records; create Feedback
Event Coordinator	Create and manage events, assign venues and vendors	Event, Client, Venue, EventVendor	Create, read, edit
Vendor	Deliver services for assigned events	Event, EventVendor	Read assigned events only
Venue Manager	Maintain venue details and availability	Venue, Event	Edit Venue; read Event
Event Administrator	Approve cancellations, manage users and reports	All objects	Full access

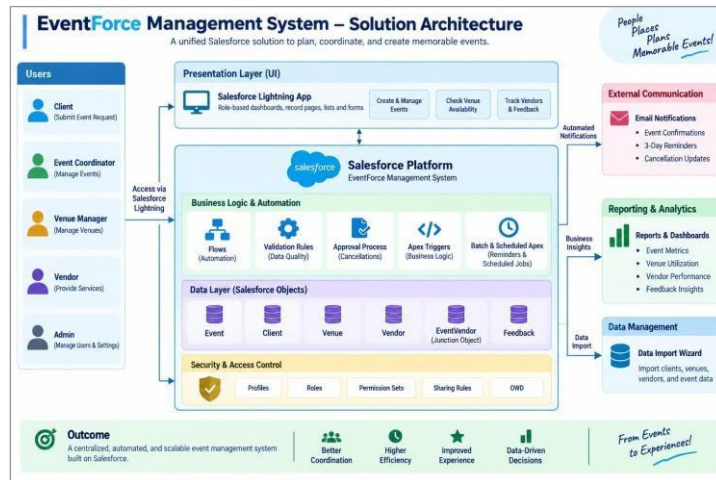


Figure 3: Stakeholder and role mapping diagram

2.1.3 Functional Scope Definition

The functional scope was divided into the following components. Each component corresponds to one or more milestones in Phase 2 to Phase 4.

Component	Planned Items
Custom objects	Event_c, Client_c, Vendor_c, Venue_c, Feedback_c, EventVendor_c
Relationships	Lookup, master-detail, many-to-many through junction object, lookup filter
Formula fields	Event_Budget__c using a CASE statement
Validation rules	Email_Valid_Address using regular expression
Approval process	Event cancellation approval with email alerts
Automation flows	Record-triggered flow for 3-day client reminder
Apex	VenueStatusHelper, PreventDoubleBooking, BatchCompleteEvents
UI	Event Planner Lightning App, custom tabs
Analytics	Report folders, Upcoming Events by Month, EventForce Operations Dashboard
Security	Profiles, roles, users, permission sets, OWD, sharing rules

2.1.4 Execution Roadmap

The roadmap follows a bottom-up approach: the data model is created first, followed by data rules, automation, user interface, data loading, security and finally testing. This order is important because every later component depends on the earlier ones - for example, a flow cannot be built before the object and fields exist, and a dashboard cannot be created before reports exist.

1. Create the Salesforce environment and the six custom objects.

2. Add tabs, fields, relationships and formula fields.
3. Apply validation rules, the approval process and the reminder flow.
4. Write Apex classes and triggers, then schedule the batch job.
5. Build the Lightning App, reports and dashboard.
6. Import test data and create profiles, roles, users, permission sets and sharing rules.
7. Perform functional, security and performance testing.
8. Prepare documentation and maintenance guidelines.

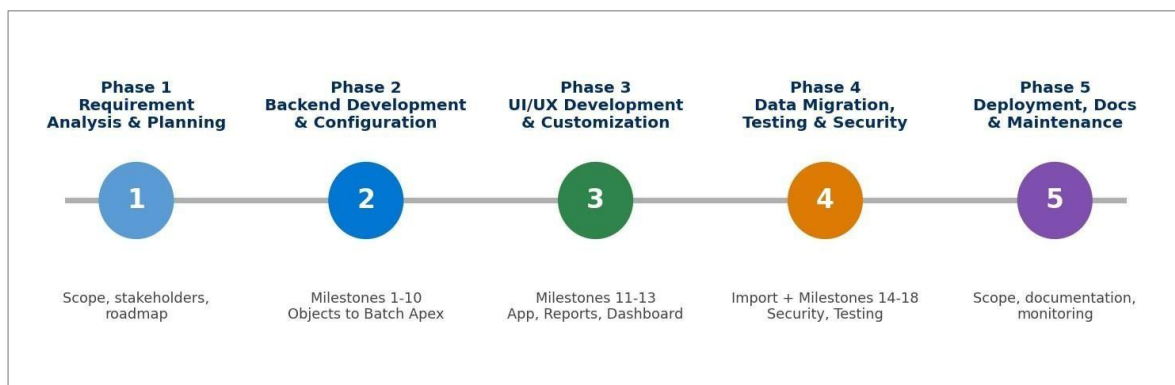


Figure 4: Project execution roadmap

2.2 Phase 2: Backend Development & Configurations

The core objective of this phase was to develop the functional engine of the application by combining declarative tools with advanced Apex business logic. Ten milestones were completed in this phase.

2.2.1 Milestone 1: Salesforce Account

A free Salesforce Developer Edition account was created at developer.salesforce.com. The Developer Edition provides a full-featured org with limited data storage, which is sufficient for learning and for academic evaluation. After signing up, a verification email was received, the password was set, and the org was opened in Lightning Experience.

1. Open the Salesforce Developer Edition sign-up page.
2. Fill in first name, last name, email, role, company, country and a unique username.
3. Accept the Master Subscription Agreement and submit the form.
4. Open the verification email and click the Verify Account link.
5. Set a new password and security question.
6. Log in and confirm that the Setup gear icon is available.

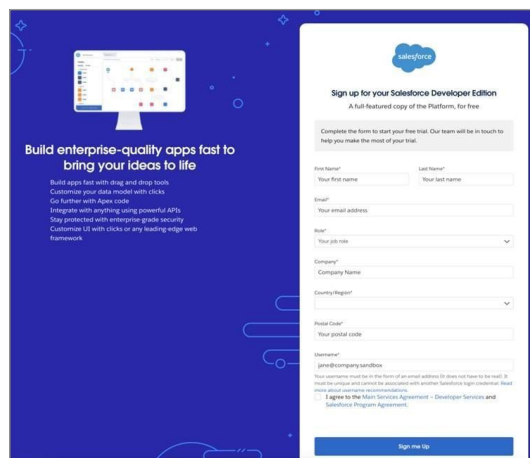
The image shows the Salesforce Developer Edition sign-up page. On the left, there's a blue sidebar with text: 'Build enterprise-quality apps fast to bring your ideas to life', followed by bullet points: 'Build apps fast with drag-and-drop tools', 'Customize your data model with clicks', 'Go further with Apex code', 'Integrate with anything using powerful APIs', 'Stay protected with enterprise-grade security', and 'Customer L4 with clicks or any leading-edge web framework'. The main content area is white and contains the sign-up form. The form title is 'Sign up for your Salesforce Developer Edition' with a subtitle 'A full-featured copy of the Platform, for free'. Below this is a grey box with the text 'Complete the form to start your free trial. Our team will be in touch to help you make the most of your trial.' The form fields are: 'First Name' and 'Last Name' (both with 'Your first name' and 'Your last name' labels), 'Email' (with 'Your email address' label), 'Role' (with 'Your job role' label and a dropdown arrow), 'Company' (with 'Company Name' label), 'Country/Region' (with a dropdown arrow), and 'Postal Code' (with 'Your postal code' label). There is a 'Username' field with the placeholder 'jane@company.com' and a note: 'Your username must be in the form of an email address (it does not have to be real). It must be unique and cannot be associated with another Salesforce login (username). Read more about username requirements.' Below the fields is a checkbox labeled 'I agree to the Master Subscription Agreement - Developer Edition and Salesforce Program Agreement'. At the bottom right is a blue 'Sign me up' button.

Figure 5: Developer Edition sign-up form

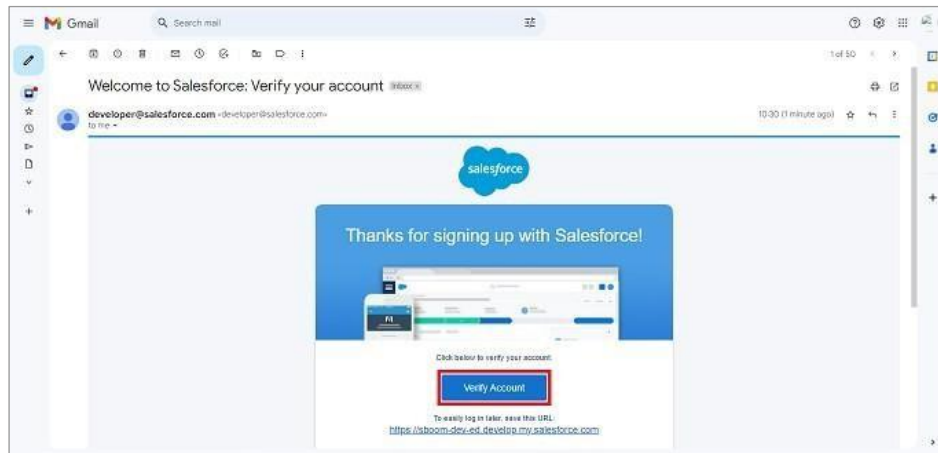
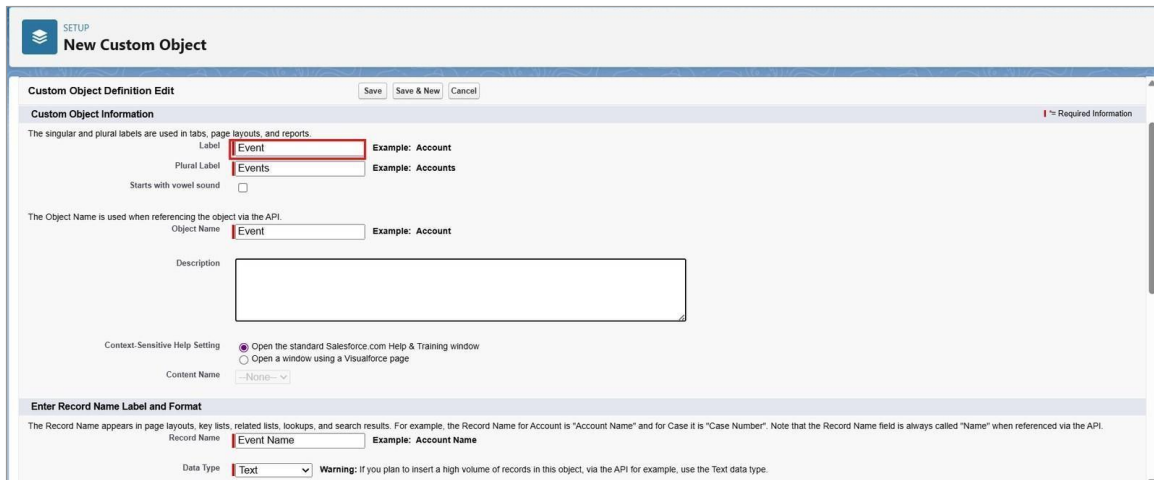


Figure 6: Account activation and first login

2.2.2 Milestone 2: Custom Objects

Objects are the database tables of Salesforce. Six custom objects were created from Setup > Object Manager > Create > Custom Object. For each object, the label, plural label, record name and data type were defined, and the options Allow Reports, Allow Activities, Track Field History and Launch New Custom Tab Wizard were enabled where required.

Object (API Name)	Purpose	Record Name
Event__c	Stores every event booking, its date, type, status and budget	Event Name (Text)
Client__c	Stores customers who book events	Client Name (Text)
Vendor__c	Stores service providers such as caterers and decorators	Vendor Name (Text)
Venue__c	Stores halls and locations, capacity and availability status	Venue Name (Text)
Feedback__c	Stores ratings and comments given after an event	Feedback Number (Auto Number)
EventVendor__c	Junction object connecting Events and Vendors (many-to-many)	EventVendor Number (Auto Number)



Custom Object Definition Edit [Save] [Save & New] [Cancel]

Custom Object Information ⓘ Required Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label: Example: Account

Plural Label: Example: Accounts

Starts with vowel sound: ☐

The Object Name is used when referencing the object via the API.

Object Name: Example: Account

Description:

Context-Sensitive Help Setting: ☒ Open the standard Salesforce.com Help & Training window
☐ Open a window using a Visualforce page

Content Name:

Enter Record Name Label and Format

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field is always called "Name" when referenced via the API.

Record Name: Example: Account Name

Data Type: Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

Figure 7: Creating the Event__c custom object



Object Manager 31+ Items, Sorted by Label

Quick Find Schema Builder Create

Label	API Name	Type	Description	Last Modified
Account	Account	Standard Object		

Custom Object Custom Object from Spreadsheet

Figure 8: Object Manager showing all six custom objects

2.2.3 Milestone 3: Tabs

A custom tab is the entry point through which users open an object. A tab was created for each object using Setup > Tabs > Custom Object Tabs > New. A tab style (colour and icon) was chosen for each object so that users can recognise them easily - for example a calendar icon for Events and a building icon for Venues. Tab visibility was set to Default On for all profiles during development and later restricted through profiles (Milestone 14).

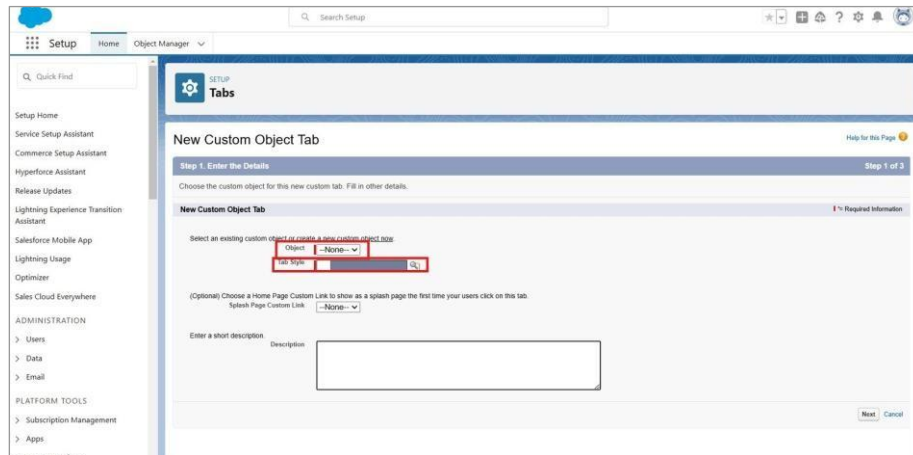


Figure 9: Creating a custom object tab

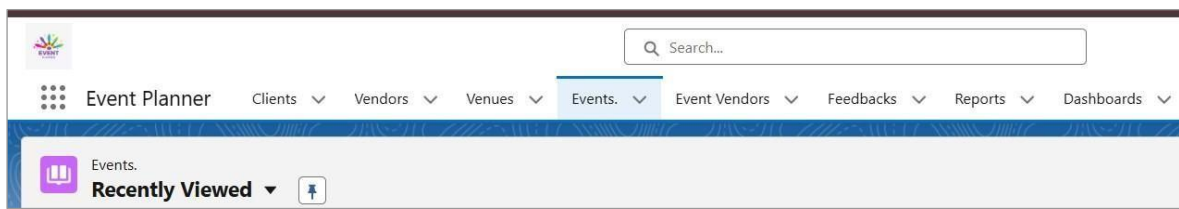


Figure 10: All custom tabs visible in the navigation bar

2.2.4 Milestone 4: Fields and Relationships

Fields define the information stored on each object. Standard data types such as Text, Number, Date, Email, Phone, Picklist and Long Text Area were used. Picklists were used wherever a fixed list of values is required, to avoid spelling mistakes and make reporting easier.

Key fields on Event__c

Field Label	API Name	Type	Notes
Event Type	Event_Type__c	Picklist	Wedding, Corporate, Conference, Birthday, Concert
Event Date	Event_Date__c	Date	Required
Status	Status__c	Picklist	Planned, Confirmed, In Progress, Completed, Cancelled
Guest Count	Guest_Count__c	Number(6,0)	Required, greater than zero
Client	Client__c	Lookup(Client)	Who booked the event

Field Label	API Name	Type	Notes
Venue	Venue__c	Lookup(Venue)	Lookup filter: only available venues
Event Budget	Event_Budget__c	Formula(Currency)	CASE statement on Event Type

Relationships

Salesforce supports several relationship types. The following were used in EventForce:

- **Lookup relationship (Event to Client and Event to Venue):** a loose relationship; the child record can exist without the parent. Chosen because an event can be saved as a draft before the final venue is confirmed.
- **Master-detail relationship (Feedback to Event):** a tight relationship in which the child record is owned by the parent. If an event is deleted, its feedback is deleted too, and feedback inherits the security of the event.
- **Junction object (EventVendor):** two master-detail relationships, one to Event and one to Vendor, create a many-to-many relationship. One event can use many vendors, and one vendor can serve many events.

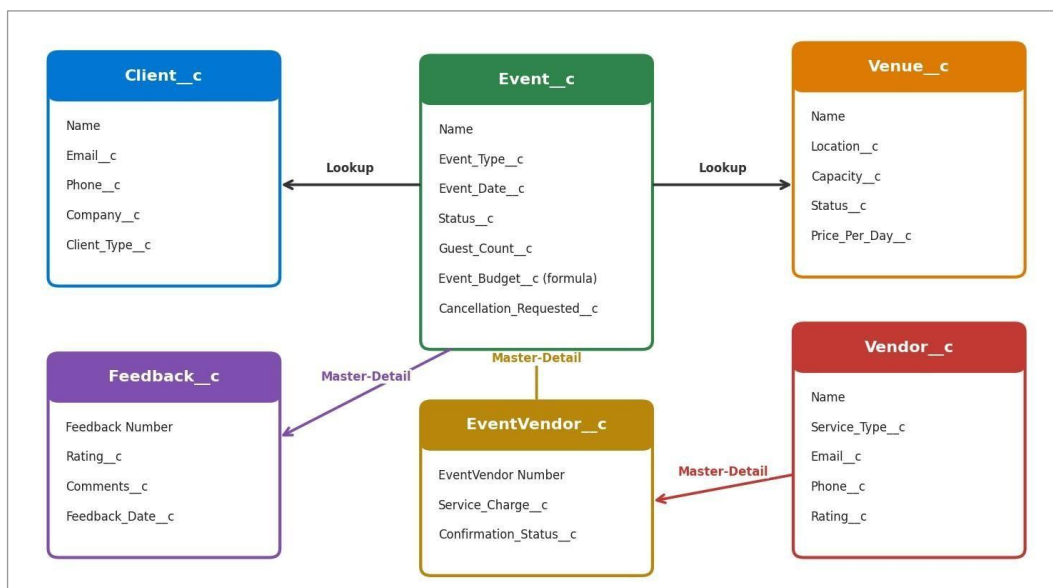


Figure 11: Relationship diagram of the EventForce data model

Formula field: Event_Budget__c

The estimated budget of an event depends on its type and the number of guests. A formula field with a CASE statement calculates the value automatically, so coordinators never need to calculate it by hand. The rates in the formula are sample values used for the prototype and can be changed by an administrator at any time.

```

CASE( TEXT(Event_Type__c),
  "Wedding",      Guest_Count__c * 1500,
  "Corporate",    Guest_Count__c * 1000,
  "Conference",   Guest_Count__c * 800,
  "Concert",      Guest_Count__c * 700,
  "Birthday",     Guest_Count__c * 500,
  Guest_Count__c * 600 )

```

Lookup filter on Event.Venue

A lookup filter limits the records a user can choose in a lookup field. On the Venue field of Event__c, a filter was added so that only venues whose Status is Available can be selected. A custom error message tells the user: "Please choose a venue that is currently available."

```

Filter Criteria : Venue__c.Status__c EQUALS Available
Filter Type      : Required
Error Message    : Please choose a venue that is currently available.

```

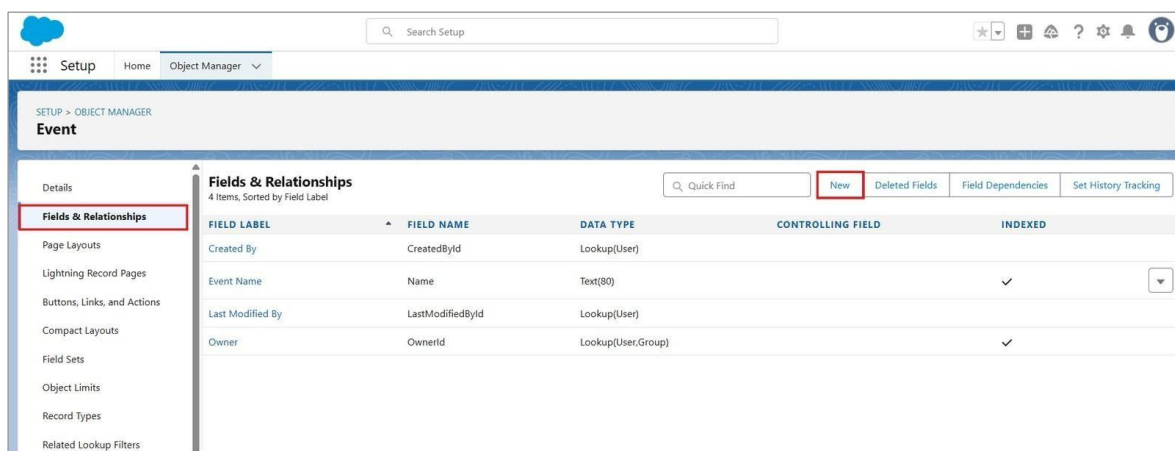


Figure 12: Fields of the Event object

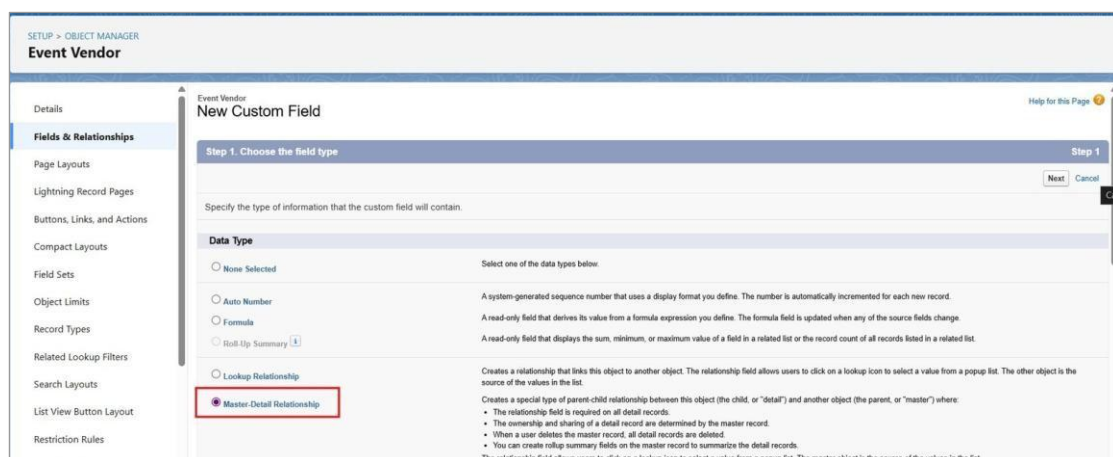


Figure 13: Master-detail relationship on Feedback__c

SETUP > OBJECT MANAGER
Event Vendor

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Search Layouts
List View Button Layout
Restriction Rules

Event Vendor
New Custom Field

Step 1. Choose the field type

Specify the type of information that the custom field will contain.

Data Type

☐ None Selected

☐ Auto Number

☐ Formula

☐ Roll-Up Summary

☒ Lookup Relationship

Master-Detail Relationship

Select one of the data types below:

A system-generated sequence number that uses a display format you define. The number is automatically incremented for each new record.

A read-only field that derives its value from a formula expression you define. The formula field is updated when any of the source fields change.

A read-only field that displays the sum, minimum, or maximum value of a field in a related list or the record count of all records listed in a related list.

Creates a relationship that links this object to another object. The relationship field allows users to click on a lookup icon to select a value from a popup list. The other object is the source of the values in the list.

Creates a special type of parent-child relationship between this object (the child, or "detail") and another object (the parent, or "master") where:

- The relationship field is required on all detail records.
- The ownership and sharing of a detail record are determined by the master record.
- When a user deletes the master record, all detail records are deleted.
- You can create rollup summary fields on the master record to summarize the detail records.
- The relationship field allows users to click on a lookup icon to select a value from a popup list. The master object is the source of the values in the list.

Figure 14: EventVendor junction object with two master-detail fields

Simple Formula Advanced Formula

Insert Field Use Formula Assistant Insert Operator

Event Budget (Currency) =

```

CASE (
  TEXT ( Event_Type__c ),
  "Wedding", 50000,
  "Corporate", 30000,
  "Birthday", 10000,
  "Anniversary", 20000,
  "Festival", 60000,
  "Concert", 40000,
  "Other", 15000,
  0
)

```

Check Syntax No syntax errors in merge fields or functions. (Compiled size: 478 characters)

Functions
-- All Function Categories --
ABS
ACOS
ADDMONTHS
AND
ASCII
ASIN
Insert Selected Function

Figure 15: Formula field Event_Budget__c

Lookup Filter

Optionally, create a filter to limit the records available to users in the lookup field. [Tell me more!](#)

▼ Hide Filter Settings

Filter Criteria Insert Suggested Criteria Clear Filter Criteria

Field	Operator	Value / Field
Feedback: Client	equals	Field Event.: Client
AND [Begin typing to search for a field...]	--None--	Value []

Add Filter Logic...

Filter Type

☒ **Required.** The user-entered value must match filter criteria.
If it doesn't, display this error message on save:
Value does not exist or does not match filter criteria.
Reset to default message

☐ **Optional.** The user can remove the filter or enter values that don't match criteria.

Lookup Window Text Add this informational message to the lookup window.
[]

Active ☒ Enable this filter

Change Field Type Save Cancel

Figure 16: Lookup filter configuration

2.2.5 Milestone 5: Validation Rules

Validation rules protect data quality by preventing a record from being saved when it does not satisfy a condition. The rule `Email_Valid_Address` was created on the `Client_c` object (and reused on `Vendor_c`) to make sure that only properly formatted email addresses are stored. This is important because reminder emails depend on a valid address.

```
Rule Name      : Email_Valid_Address
Active         : Yes
Error Formula  :
AND(
  NOT(ISBLANK(Email__c)),
  NOT(REGEX(Email__c, "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"))
)
Error Message  : Please enter a valid email address, e.g. name@example.com
Error Location : Field - Email
```

Regular expressions describe text patterns. In the rule above, the part before the `@` symbol allows letters, digits and a few symbols; the part after `@` contains the domain name; and the last part requires a dot followed by at least two letters. If the pattern does not match, the rule returns TRUE and Salesforce blocks the save and shows the error message.

Test Input	Expected Result	Outcome
meera@eventforce.com	Saved	Pass
meera@eventforce	Error message shown	Pass
meera.eventforce.com	Error message shown	Pass
(blank)	Saved (field not required)	Pass

SETUP

Validation Rule **Email_Valid_Address**

Object	Client__c (reused on Vendor__c)
Active	Yes
Error Condition Formula	AND(NOT(ISBLANK(Email__c)), NOT(REGEX(Email__c, "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}")))
Error Message	Please enter a valid email address, e.g. name@example.com
Error Location	Field - Email

Figure 17: Validation rule definition

CLIENT
New Client

Client Name
Meera Events

* Email
meera@eventforce

Please enter a valid email address, e.g. name@example.com

Cancel Save

We hit a snag.
Review the errors on this page.
- Please enter a valid email address

Figure 18: Validation rule in action

2.2.6 Milestone 6: Approval Process

An approval process automates the way a record is approved by one or more people. In EventForce, cancelling an event is a sensitive action because it affects the client, the venue and the vendors. Therefore a cancellation approval process was created on Event__c. A coordinator cannot cancel an event directly; the request goes to the Event Administrator, who approves or rejects it.

Configuration summary

Setting	Value
Process name	Event_Cancellation_Approval
Object	Event__c
Entry criteria	Cancellation_Requested__c = TRUE (a checkbox set by the coordinator)
Approver	Event Administrator (role / user)
Initial submission actions	Lock the record; email alert to the Event Administrator
Final approval actions	Status = Cancelled; unlock; email alert to Client and Coordinator
Final rejection actions	Cancellation_Requested__c = FALSE; unlock; email alert to the Coordinator
Record editability	Administrators can edit locked records

Email alerts use email templates that contain merge fields such as the event name, date and client name, so that every message is clear and personalised. Using templates also means the wording can be changed later without editing the process.

1. Create email templates for Submission, Approval and Rejection.

2. Create email alerts that use each template and set the recipients.
3. Create field updates for Status and the cancellation checkbox.
4. Create the approval process using the Standard Setup Wizard.
5. Define entry criteria, approver and actions.
6. Activate the approval process.
7. Add the Approval History related list to the Event page layout.

Figure 19: Approval process definition

Figure 20: Email alerts and templates

Approval Processes

Event.: Event_Cancellation_Process

« Back to Approval Process List

Process Definition Detail

Process Name: Event_Cancellation_Process

Unique Name: Event_Cancellation_Process

Description: Event.: Event Status EQUALS Pending Cancellation

Entry Criteria: Administrator OR Current Approver

Record Editability: Administrator OR Current Approver

Approval Assignment Email Template: Event_Cancellation_Request_Notification

Initial Submitters: Event. Owner

Created By: Sushanth Sush, 9/22/2025, 11:12 PM

Modified By: Sushanth Sush, 9/23/2025, 9:47 AM

Buttons: Edit, Clone, Delete, **Activate**

Figure 21: Submit for Approval and pending status

Email Template Information

Folder: Unfiled Public Classic Email Templates

Available For Use: ☒

Email Template Name: Event Cancellation Request

Template Unique Name: Event_Cancellation_Request

Encoding: Unicode (UTF-8)

Description:

Subject: Approval Request: Cancel Event (!Event__c.Name)

Email Body:

Hello,

An approval request has been submitted to cancel the event:

- Event Name: (!Event__c.Name)
- Event Date: (!Event__c.Event_Date__c)

Please review and take action:

Thank you,
EventForce Management System

Buttons: Save, Save & New, Cancel

Figure 22: Approval email received by the Administrator

2.2.7 Milestone 7: Flows

Salesforce Flow is the main declarative automation tool. A Record-Triggered Flow runs automatically when a record is created or updated. In EventForce a flow was built to send a reminder email to the client three days before the event date, so that the coordinator does not need to remember it manually.

Flow design

Element	Configuration
Flow name	Event_Reminder_3_Days
Type	Record-Triggered Flow (Autolaunched)
Object	Event__c, trigger when a record is created or updated
Entry condition	Status__c equals Confirmed

Element	Configuration
Scheduled path	3 Days Before Event__Date__c
Get Records	Retrieve the related Client__c to read the client email and name
Action	Send Email: subject "Reminder: your event is in 3 days", body with event name, date and venue
Fault path	Sends an email to the administrator if the action fails

The scheduled path is an important feature of record-triggered flows. Instead of running immediately, the flow waits until the specified time relative to a date field on the record. If the event date is changed later, Salesforce recalculates the scheduled time automatically, so the client always receives the reminder at the correct time.

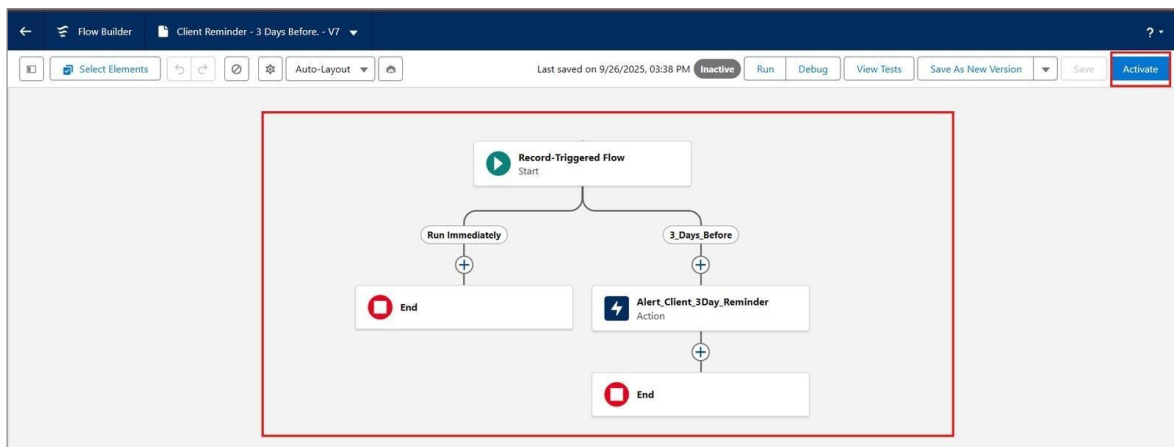


Figure 23: Flow canvas of Event_Reminder_3_Days

The 'Configure Scheduled Paths' dialog shows the configuration for a scheduled path. The 'SCHEDULED PATHS' section lists 'Run Immediately' and '3_Days_Before'. The '3_Days_Before' path is selected, and its details are shown in the 'SCHEDULED PATH DETAILS' section. The details include: Path Label (3_Days_Before), API Name (X3_Days_Before), Time Source (Event__c: Event Date), Offset Number (3), and Offset Options (Days Before). A 'Delete Path' button is also visible.

Figure 24: Start element and scheduled path settings

Figure 25: Send Email action configuration



Figure 26: Flow activated

2.2.8 Milestone 8 and 9: Apex Classes and Triggers

Some requirements are too complex for declarative tools. Checking whether two events overlap on the same venue, and keeping the venue status synchronised with its events, require querying related records and comparing them in bulk. Apex, the Java-like programming language of Salesforce, was used for these requirements. Two components were written: the **VenueStatusHelper** class and the **PreventDoubleBooking** trigger.

Design principles followed

- **Bulkification:** code handles up to 200 records in one transaction without running a query or DML statement inside a loop.
- **Separation of logic:** the trigger contains almost no logic; it calls the helper class, which makes the code easier to test and reuse.

- **Governor limits:** SOQL queries are done once per transaction and results are stored in maps and sets.
- **User friendly errors:** the addError method shows a clear message directly on the record page.

VenueStatusHelper class

This class has two responsibilities. The first is to detect conflicts: it returns the set of venue-and-date combinations that are already booked. The second is to update the status of venues to Booked or Available depending on whether they have active events.

```
public with sharing class VenueStatusHelper {

    // Returns existing active events grouped by Venue Id
    public static Map<Id, List<Event__c>> getActiveEventsByVenue(Set<Id>
venueIds) {
        Map<Id, List<Event__c>> result = new Map<Id, List<Event__c>>();
        for (Event__c e : [
            SELECT Id, Name, Venue__c, Event_Date__c, Status__c
            FROM Event__c
            WHERE Venue__c IN :venueIds
            AND Status__c IN ('Planned','Confirmed','In Progress')
        ]) {
            if (!result.containsKey(e.Venue__c)) {
                result.put(e.Venue__c, new List<Event__c>());
            }
            result.get(e.Venue__c).add(e);
        }
        return result;
    }

    // Marks venues Booked or Available based on their active events
    public static void updateVenueStatus(Set<Id> venueIds) {
        if (venueIds == null || venueIds.isEmpty()) return;
        Map<Id, List<Event__c>> active = getActiveEventsByVenue(venueIds);
        List<Venue__c> toUpdate = new List<Venue__c>();
        for (Id vId : venueIds) {
            Venue__c v = new Venue__c(Id = vId);
            v.Status__c = active.containsKey(vId) ? 'Booked' : 'Available';
            toUpdate.add(v);
        }
        update toUpdate;
    }
}
```

PreventDoubleBooking trigger

The trigger runs before insert and before update on Event__c. For each event being saved it checks whether another active event exists on the same venue and the same date. If a conflict is found, the save is blocked using addError. After a successful save, an after-trigger section refreshes the venue status through the helper class.

```

trigger PreventDoubleBooking on Event__c (before insert, before update, after
insert, after update, after delete) {

    if (Trigger.isBefore) {
        Set<Id> venueIds = new Set<Id>();
        for (Event__c e : Trigger.new) {
            if (e.Venue__c != null) venueIds.add(e.Venue__c);
        }
        Map<Id, List<Event__c>> existing =
VenueStatusHelper.getActiveEventsByVenue(venueIds);

        for (Event__c e : Trigger.new) {
            if (e.Venue__c == null || e.Event_Date__c == null) continue;
            if (e.Status__c == 'Cancelled' || e.Status__c == 'Completed')
continue;
            if (!existing.containsKey(e.Venue__c)) continue;
            for (Event__c other : existing.get(e.Venue__c)) {
                if (other.Id != e.Id && other.Event_Date__c ==
e.Event_Date__c) {
                    e.addError('This venue is already booked on ' +
                                e.Event_Date__c.format() + ' for event: ' +
                                other.Name);
                }
            }
        }
    }

    if (Trigger.isAfter) {
        Set<Id> venueIds = new Set<Id>();
        List<Event__c> records = Trigger.isDelete ? Trigger.old :
Trigger.new;
        for (Event__c e : records) {
            if (e.Venue__c != null) venueIds.add(e.Venue__c);
        }
        if (Trigger.isUpdate) {
            for (Event__c e : Trigger.old) {
                if (e.Venue__c != null) venueIds.add(e.Venue__c);
            }
        }
        VenueStatusHelper.updateVenueStatus(venueIds);
    }
}

```

Note that conflicts inside the same batch of records (for example, two new events with the same venue and date uploaded together) are also a possibility in bulk loads; the data import in Phase 4 was prepared so that the sample data has no such overlaps.

Test class

Salesforce requires at least 75 percent code coverage before Apex can be deployed to production. A test class was written to cover the helper class and the trigger, including both positive and negative scenarios.

```

@isTest
private class PreventDoubleBookingTest {
    @isTest static void blocksSecondEventOnSameDate() {

```

```

        Venue__c v = new Venue__c(Name = 'Grand Hall', Capacity__c = 300,
Status__c = 'Available');
        insert v;
        Event__c e1 = new Event__c(Name = 'Event A', Venue__c = v.Id,
            Event_Date__c = Date.today().addDays(10), Status__c =
'Confirmed',
            Event_Type__c = 'Wedding', Guest_Count__c = 200);
        insert e1;
        Event__c e2 = new Event__c(Name = 'Event B', Venue__c = v.Id,
            Event_Date__c = Date.today().addDays(10), Status__c = 'Planned',
            Event_Type__c = 'Corporate', Guest_Count__c = 100);
        Test.startTest();
        Database.SaveResult sr = Database.insert(e2, false);
        Test.stopTest();
        System.assert(!sr.isSuccess(), 'Second booking should fail');
        System.assertEquals('Booked', [SELECT Status__c FROM Venue__c WHERE
Id = :v.Id].Status__c);
    }
}

```

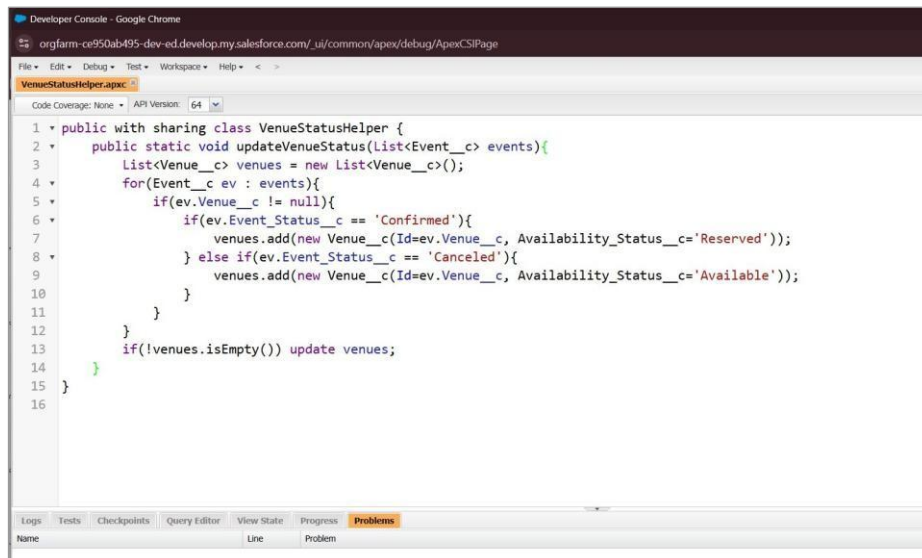


Figure 27: VenueStatusHelper class in Developer Console

```

1  trigger PreventDoubleBooking on Event__c (before insert, before update) {
2      Set<Id> venueIds = new Set<Id>();
3      for(Event__c ev : Trigger.new){
4          if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
5      }
6
7      Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
8      for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
9                          FROM Event__c WHERE Venue__c IN :venueIds]){
10         if(!venueEventMap.containsKey(e.Venue__c)){
11             venueEventMap.put(e.Venue__c, new List<Event__c>());
12         }
13         venueEventMap.get(e.Venue__c).add(e);
14     }
15
16     for(Event__c ev : Trigger.new){
17         if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
18             for(Event__c existing : venueEventMap.get(ev.Venue__c)){
19                 if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
20                     ev.addError('This Venue is already booked on this date.');

```

Figure 28: PreventDoubleBooking trigger

```

1  trigger EventTrigger13 on Event__c (after insert, after update) {
2      VenueStatusHelper.updateVenueStatus(Trigger.new);
3  }

```

Figure 29: Test class run and code coverage

```

1  trigger PreventDoubleBooking on Event__c (before insert, before update) {
2      Set<Id> venueIds = new Set<Id>();
3      for(Event__c ev : Trigger.new){
4          if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
5      }
6
7      Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
8      for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
9                          FROM Event__c WHERE Venue__c IN :venueIds]){
10         if(!venueEventMap.containsKey(e.Venue__c)){
11             venueEventMap.put(e.Venue__c, new List<Event__c>());
12         }
13         venueEventMap.get(e.Venue__c).add(e);
14     }
15
16     for(Event__c ev : Trigger.new){
17         if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
18             for(Event__c existing : venueEventMap.get(ev.Venue__c)){
19                 if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
20                     ev.addError('This Venue is already booked on this date.');

```

Figure 30: Double booking blocked on the Event page

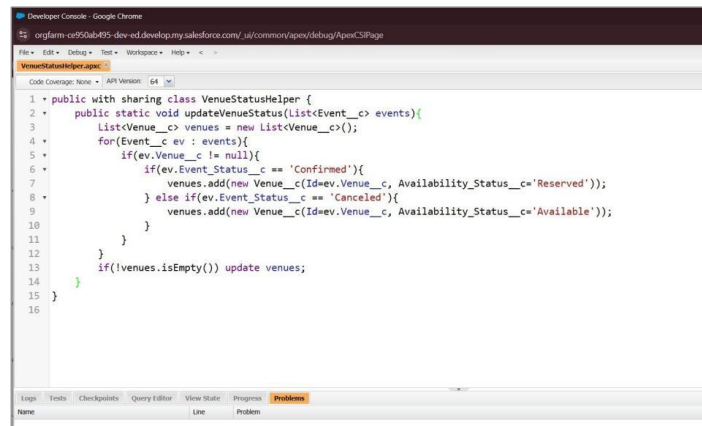


Figure 31: Venue status automatically changed to Booked

2.2.9 Milestone 10: Asynchronous Apex

Asynchronous Apex runs in the background, outside the normal user transaction. It is ideal for work that touches many records or must run at a fixed time. For EventForce, past events must be marked Completed every night so that reports, venue availability and the dashboard remain correct without manual effort.

Two features were combined in a single class named **BatchCompleteEvents**: Batch Apex (Database.Batchable) processes records in chunks of up to 200 so that governor limits are never exceeded even for large volumes, and Schedulable Apex (Schedulable) lets the job run automatically at a chosen time.

```
global class BatchCompleteEvents implements Database.Batchable<SObject>,
Schedulable {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Venue__c
            FROM Event__c
            WHERE Event_Date__c < TODAY
            AND Status__c IN ('Planned','Confirmed','In Progress')
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Event__c> scope) {
        Set<Id> venueIds = new Set<Id>();
        for (Event__c e : scope) {
            e.Status__c = 'Completed';
            if (e.Venue__c != null) venueIds.add(e.Venue__c);
        }
        update scope;
        VenueStatusHelper.updateVenueStatus(venueIds);
    }
}
```

```

global void finish(Database.BatchableContext bc) {
    System.debug('BatchCompleteEvents finished.');
```

```

}

// Schedulable entry point
global void execute(SchedulableContext sc) {
    Database.executeBatch(new BatchCompleteEvents(), 200);
}
}

```

Scheduling the job

The job was scheduled to run every night at 12:30 AM using a CRON expression. It can be scheduled either from Setup > Apex Classes > Schedule Apex or by running the following statement in the Developer Console Execute Anonymous window.

```

// Seconds Minutes Hours DayOfMonth Month DayOfWeek
System.schedule('Nightly Complete Events', '0 30 0 * * ?', new
BatchCompleteEvents());

```

CRON Part	Value	Meaning
Seconds	0	At second 0
Minutes	30	At minute 30
Hours	0	At 12 AM
Day of month	*	Every day
Month	*	Every month
Day of week	?	No specific day

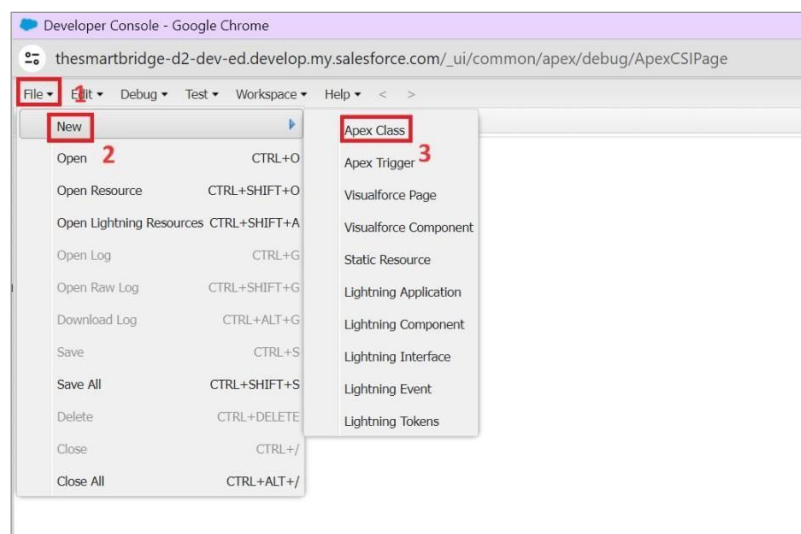


Figure 32: BatchCompleteEvents class

Apex Classes

Save Cancel

Job Name: **Daily Event Completion**

Apex Class: **ScheduleCompleteEvents**

Schedule Using: ☒ Schedule Builder ☐ Cron Expression

Schedule Apex Execution

Frequency: ☒ Weekly ☐ Monthly

Recurs every week on:

- ☒ Sunday
- ☒ Monday
- ☒ Tuesday
- ☒ Wednesday
- ☒ Thursday
- ☒ Friday
- ☒ Saturday

Start: 9/26/2025 (9/26/2025)

End: 9/26/2025 (9/26/2025)

Preferred Start Time: 8:00 PM

Exact start time will depend on job queue activity.

Figure 33: Schedule Apex page

Apex Classes

Apex Code is an object oriented programming language that allows developers to develop on-demand business applications on the Lightning Platform.

Percent of Apex Used: 0.04%
You are currently using 2,625 characters of Apex Code (excluding comments and @isTest annotated classes) in your organization, out of an allowed limit of 6,000,000 characters. Note that the amount in use includes both Apex Classes and Triggers defined in your organization.

Estimate your organization's code coverage

Compile all classes

View: All Create New View

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other All

Action	Name	Namespace Prefix	Api Version	Status	Size Without Comments	Last Modified By	Has Trace Flags
Edit Del Security	BatchCompleteEvents		64.0	Active	651	Sushanth.Sush	9/26/2025, 6:23 AM
Edit Del Security	ClientEventCountHelper		64.0	Active	40	Sushanth.Sush	9/26/2025, 5:30 AM
Edit Del Security	ScheduleCompleteEvents		64.0	Active	180	Sushanth.Sush	9/26/2025, 6:24 AM
Edit Del Security	VenueStatusHelper		64.0	Active	643	Sushanth.Sush	9/26/2025, 5:45 AM

Figure 34: Scheduled Jobs list

Setup Home Object Manager

Search Setup

Apex Classes

Didn't find what you're looking for? Try using Global Search.

Home

Create

Data Cloud
Connect, prepare, harmonize, unify, and analyze data to get a 360-degree view of your customers.

Get Started with Einstein Bots
Launch an AI-powered bot to automate your digital connections.

Mobile Publisher
Use the Mobile Publisher to create your own branded mobile app.

Figure 35: Apex Jobs completed

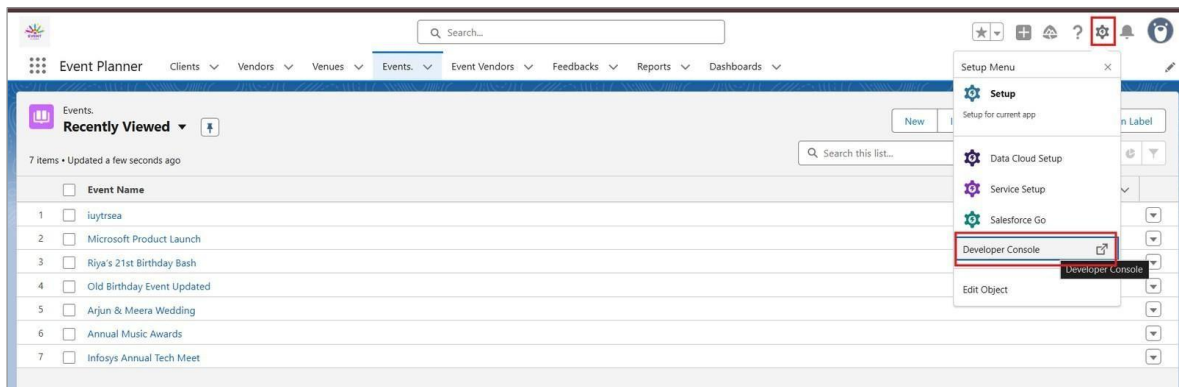


Figure 36: Past events changed to Completed

2.3 Phase 3: UI/UX Development & Customization

The core objective of this phase was to design a responsive interface and analytics dashboards that give users operational visibility. After the backend was complete, the user experience was organised into a dedicated application, reports and a dashboard.

2.3.1 Milestone 11: Lightning App

A Lightning App is a collection of tabs and utilities that gives users all the tools they need for a job in one place. A custom app named **Event Planner** was created from Setup > App Manager > New Lightning App. The app uses the standard navigation style, an EventForce theme colour and a company logo.

Setting	Value
App name / API name	Event Planner / Event_Planner
Navigation style	Standard navigation
Navigation items	Home, Events, Clients, Vendors, Venues, Feedback, Reports, Dashboards
Utility items	Recent Items, Notes
User profiles	Event Admin, Event Coordinator, Vendor Manager, Client Profile
Home page	Customised with upcoming events list and dashboard component

Custom list views were created on the Event tab, such as "Upcoming Events", "Confirmed Events" and "My Events", and the compact layout and page layouts were arranged so that the most important fields are visible at the top of every record. Highlight panels, related lists for EventVendor and Feedback, and the Approval History were added to the Event page.

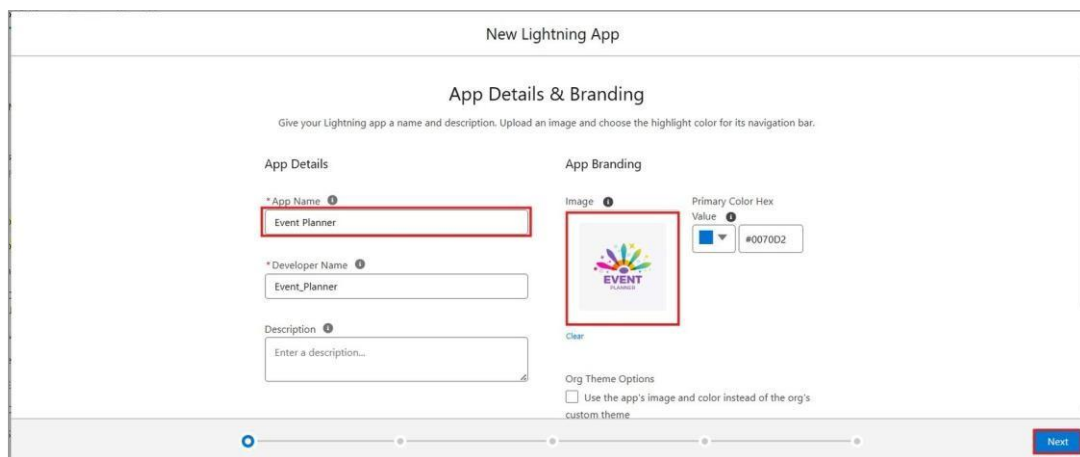


Figure 37: Lightning App creation - app details

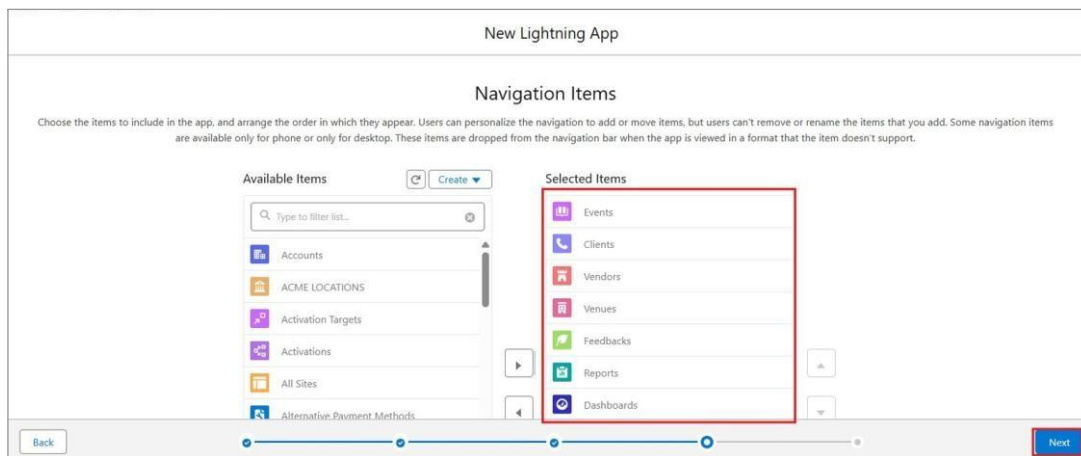


Figure 38: Navigation items selection

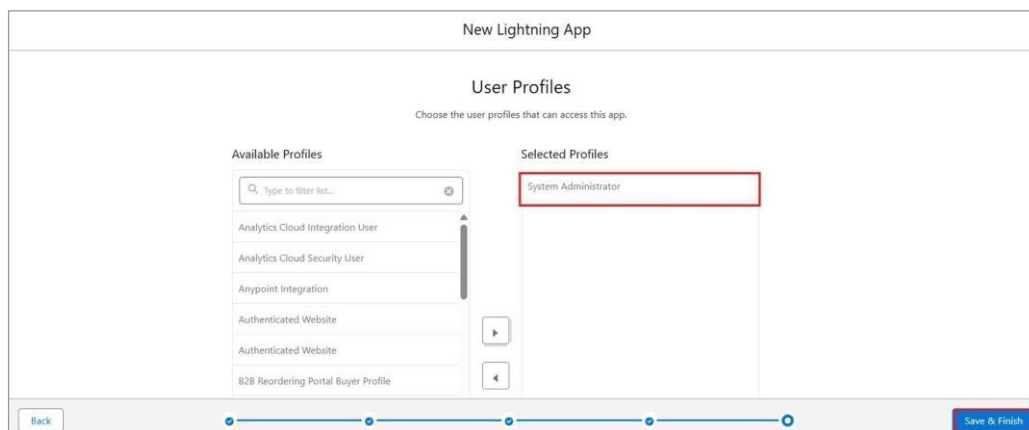


Figure 39: User profile assignment to the app

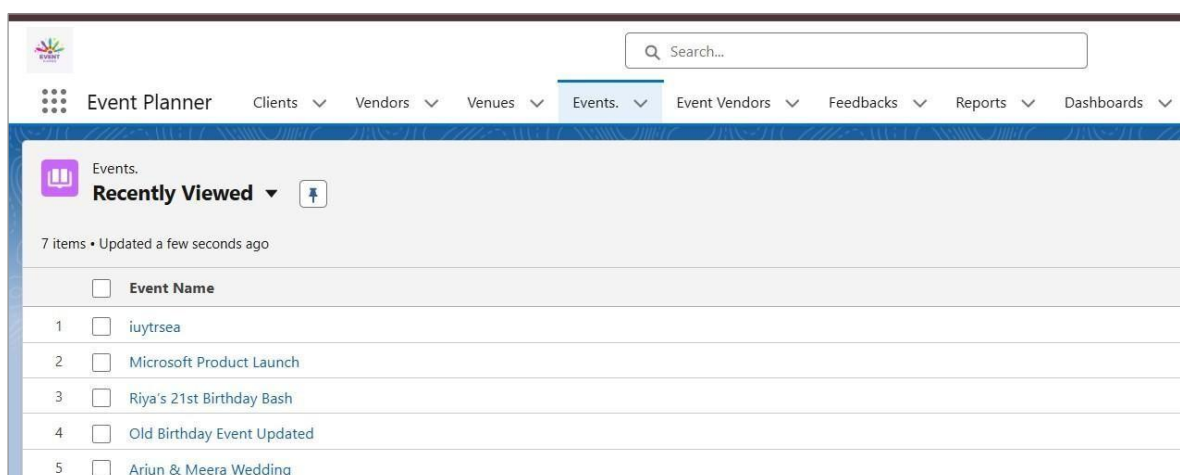


Figure 40: Event Planner app home page

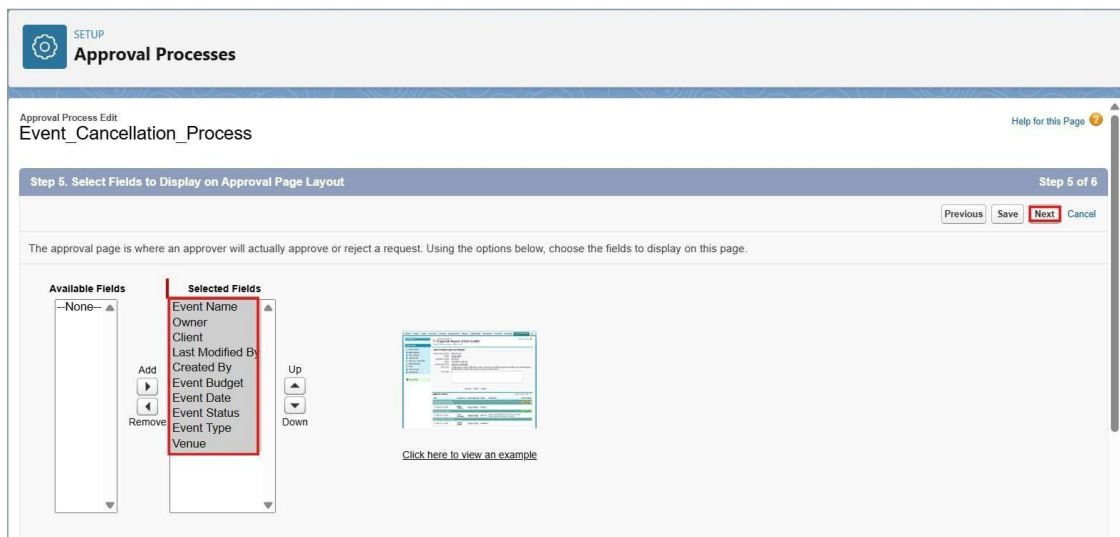


Figure 41: Event record page with related lists

2.3.2 Milestone 12: Reports

Reports let users analyse data without writing queries. A report folder named **EventForce Reports** was created, and sharing was configured so that the Event Admin has Manager access, Event Coordinators have Viewer access, and other roles see only the reports meant for them. Using folders for sharing is more efficient than sharing each report individually.

Report	Type	Grouping / Filters	Purpose
Upcoming Events by Month	Events (Tabular + Summary)	Group by Event Date (Calendar Month); Status not Cancelled; Date >= Today	Plan resources for coming months
Events by Status	Summary	Group by Status	Track pipeline of events
Events with Vendors	Events with EventVendors	Group by Vendor	Vendor workload
Venue Utilization	Venues with Events	Group by Venue	Identify most-used venues
Feedback by Event Type	Events with Feedback	Group by Event Type; average rating	Measure client satisfaction

1. Create custom report types if related objects are required (e.g. Events with EventVendors).
2. Open Reports > New Report and choose the report type.
3. Add columns: Event Name, Client, Venue, Event Date, Status, Budget.
4. Add filters and group rows by Event Date (calendar month).

- 5. Add the Sum of Event Budget summary field.
- 6. Save the report in the EventForce Reports folder.
- 7. Share the folder with roles and set Viewer / Manager access.

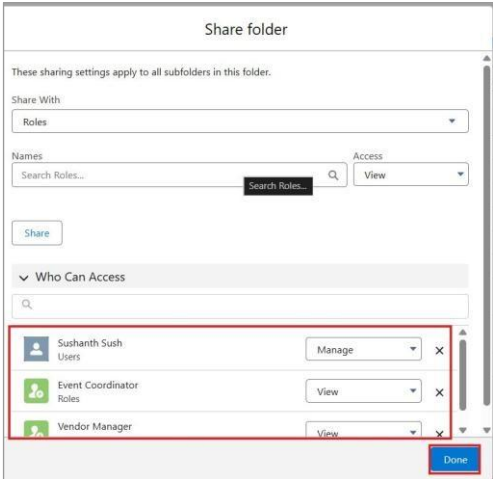


Figure 42: Report folder creation and sharing

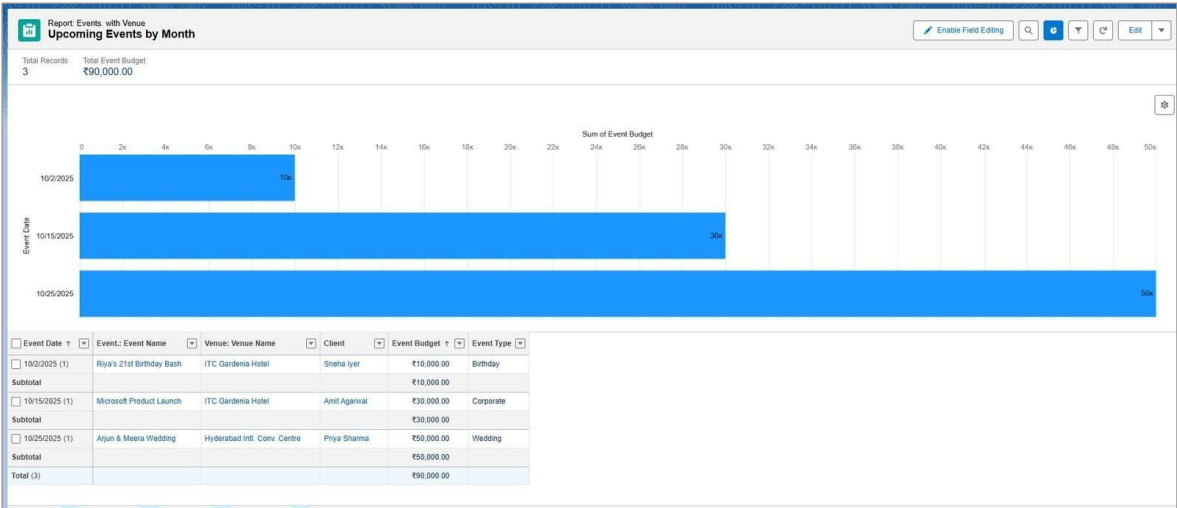


Figure 43: Upcoming Events by Month report

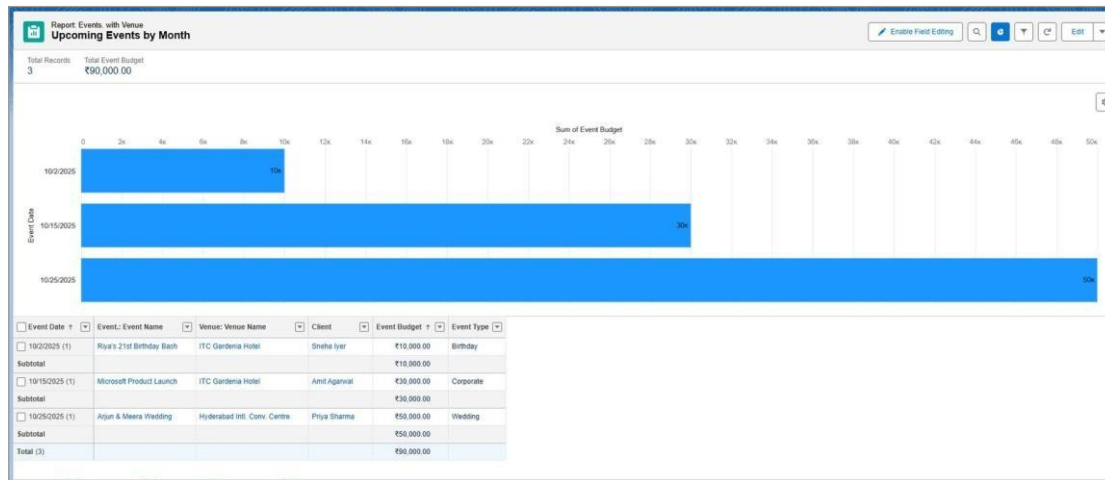


Figure 44: Report filters and columns

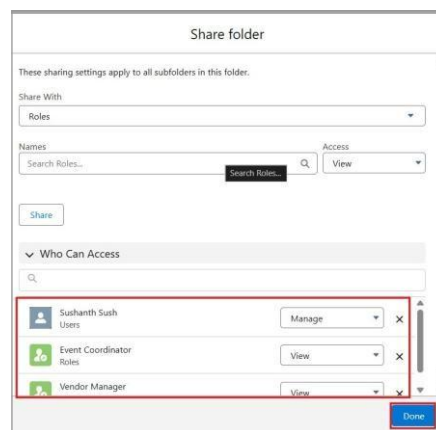


Figure 45: Reports folder listing

2.3.3 Milestone 13: Dashboards

A dashboard is a visual representation of one or more reports. The **EventForce Operations Dashboard** was built to give management a single view of event operations. Each component is based on a report and uses a suitable chart type so that the message is clear at a glance.

Component	Chart Type	Source Report	Insight
Events by Status	Donut chart	Events by Status	Share of planned, confirmed, completed and cancelled events
Upcoming Events by Month	Column chart	Upcoming Events by Month	Future workload
Top Vendors	Horizontal bar chart	Events with Vendors	Most-used vendors
Venue Utilization	Table	Venue Utilization	Busiest venues
Average Feedback Rating	Gauge	Feedback by Event Type	Overall satisfaction

The dashboard folder is shared with the same roles as the report folder, and the dashboard is refreshed automatically each morning. The running user was set to a user with full visibility so that every viewer sees the complete picture of operations.

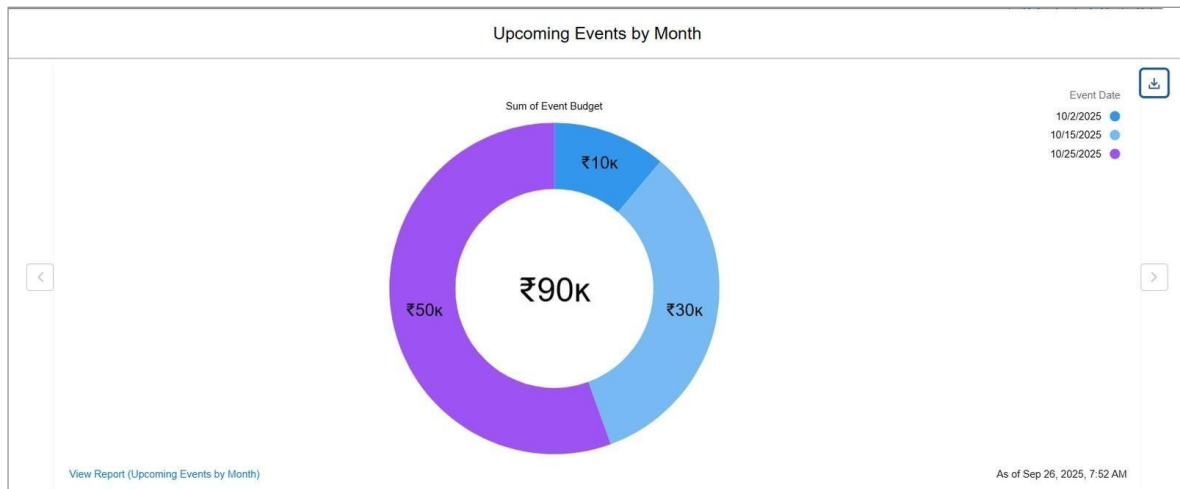


Figure 46: Dashboard builder - adding components



Figure 47: Component settings

2.4 Phase 4: Data Migration, Testing & Security

The core objective of this phase was to secure data access, load pre-validated test data and perform system testing. Realistic data is needed so that reports, dashboards, flows and security rules can be verified, and security is needed before any user other than the administrator can use the system.

2.4.1 Data Migration

Data was imported through the **Data Import Wizard**, a guided tool available in Setup that supports standard and custom objects and up to 50,000 records at a time. Data was prepared in CSV files, cleaned, and validated against the validation rules before loading. The import order followed the relationship order: parent records first and child records afterwards, so that lookup fields could be matched.

Order	Object	Records (sample)	Matching Field
1	Client__c	[number]	Name
2	Venue__c	[number]	Name
3	Vendor__c	[number]	Name
4	Event__c	[number]	Client and Venue matched by Name
5	Feedback__c	[number]	Event matched by Name

1. Prepare CSV files with column headers that match field labels.
2. Remove duplicates and check date format and picklist values.
3. Open Setup > Data Import Wizard > Launch Wizard.
4. Choose Custom Objects, select the object and the action (Add New Records).
5. Upload the CSV file and map each CSV column to a Salesforce field.
6. Start the import and review the result email.
7. Open the object tab and verify the records and any errors.

SETUP

Data Import Wizard

1 Choose data

2 Upload CSV

3 Edit mapping

4 Start import

What kind of data are you importing?

Standard Objects

Custom Objects

> Event
Client
Vendor
Venue
Feedback

What do you want to do?
☒ Add new records
☐ Update existing records

Figure 49: Data Import Wizard - choose object

SETUP

Data Import Wizard

1 Choose data

2 Upload CSV

3 Edit mapping

4 Start import

Edit field mapping

CSV Header	Salesforce Field
Event Name	Name
Event Date	Event Date
Event Type	Event Type
Event Status	Event Status
Client	Client (matched by Name)
Venue	Venue (matched by Name)

Figure 50: Field mapping step

SETUP

Data Import Wizard

1 Choose data

2 Upload CSV

3 Edit mapping

4 Start import

Import completed

Your data import request has been successfully submitted.

Object:

Event__c

Action:

Add new records

Status:

Completed

Errors:

0

Figure 51: Import completed

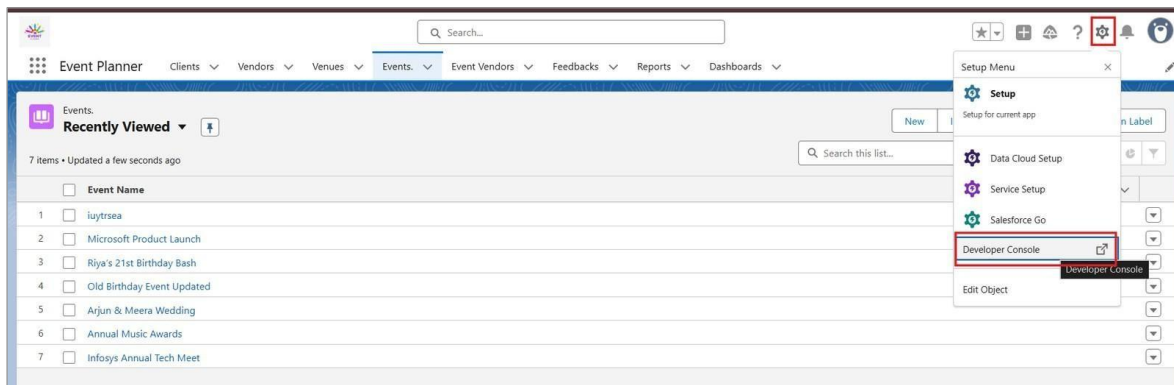


Figure 52: Imported event records

2.4.2 Milestone 14: Profiles

A profile controls what a user can do: which objects they can access, which fields they can see, which apps and tabs they can open and which system permissions they have. Four custom profiles were created by cloning an existing profile and then adjusting permissions to match the stakeholder map.

Object	Event Admin	Event Coordinator	Vendor Manager	Client Profile
Event__c	CRUD + View All	Create, Read, Edit	Read	Read
Client__c	CRUD + View All	Create, Read, Edit	No access	Read
Vendor__c	CRUD + View All	Read	Create, Read, Edit	No access
Venue__c	CRUD + View All	Read	Read	Read
Feedback__c	CRUD + View All	Read	No access	Create, Read
EventVendor__c	CRUD + View All	Create, Read, Edit	Read, Edit	No access

The screenshot shows the 'Clone Profile' setup screen in Salesforce. It prompts the user to 'Enter the name of the new profile.' Below this, a message states 'You must select an existing profile to clone from.' The 'Existing Profile' dropdown is set to 'System Administrator', 'User License' is 'Salesforce', and 'Profile Name' is 'Event Admin'. The 'Save' button is highlighted.

Figure 53: Creating a custom profile

Custom Object Permissions							
	Basic Access				Data Administration		
	Read	Create	Edit	Delete	View All Records	Modify All Records	View All Fields
ACME LOCATIONS	☐	☐	☐	☐	☐	☐	☐
Clients	☒	☐	☒	☐	☐	☐	☐
Events	☒	☒	☒	☒	☐	☐	☐
Event Vendors	☐	☐	☐	☐	☐	☐	☐

	Basic Access				Data Administration		
	Read	Create	Edit	Delete	View All Records	Modify All Records	View All Fields
Feedbacks	☒	☒	☐	☐	☐	☐	☐
Vendors	☒	☐	☐	☐	☐	☐	☐
Venues	☒	☐	☐	☐	☐	☐	☐

Figure 54: Object permissions in a profile

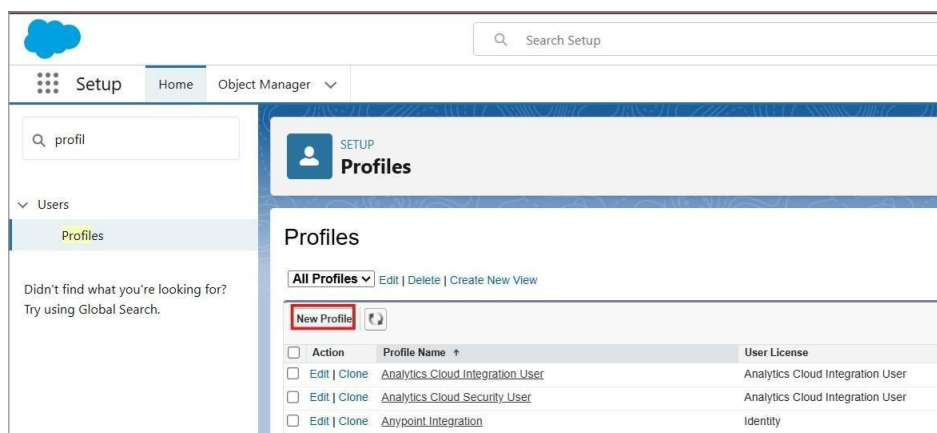


Figure 55: List of the four custom profiles

2.4.3 Milestone 15: Roles

Roles define the record-level visibility hierarchy. Users in a higher role can see the records owned by users below them. The Event Admin role was placed at the top of the hierarchy so that the administrator can see all data. The Event Coordinator and Vendor Manager roles sit below it, and the Client role is placed at the lowest level.

```
Event Admin
|-- Event Coordinator
|-- Vendor Manager
    |-- Client
```

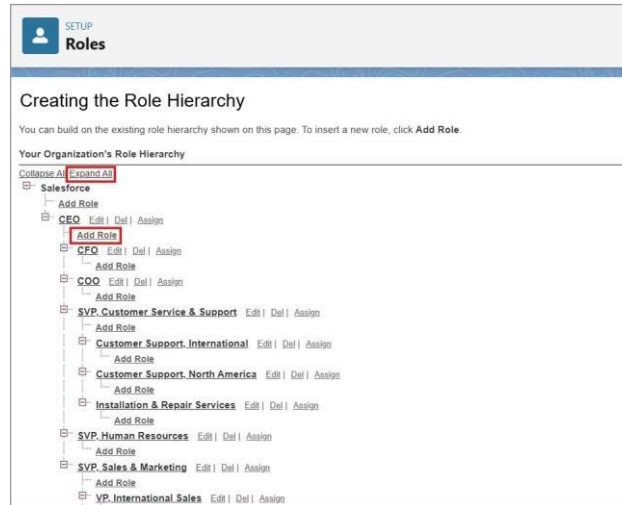


Figure 56: Role hierarchy

2.4.4 Milestone 16: Users

Users were created for each role so that the system could be tested from every point of view. Each user was given a unique username in email format, a profile and a role. A user without a profile and role cannot sign in or see any data.

User	Profile	Role	Purpose
[Admin user]	Event Admin	Event Admin	System administration and approvals
[Coordinator user]	Event Coordinator	Event Coordinator	Create and manage events
[Vendor manager user]	Vendor Manager	Vendor Manager	Maintain vendors
[Client user]	Client Profile	Client	Verify limited client access

The screenshot shows the 'New User' form in Salesforce. The form is titled 'New User' and has a 'User Edit' section. The 'General Information' section includes fields for First Name (Sizzler), Last Name (Jaligam), Alias (sjali), Email (sushanth2805@gmail.com), Username (sizzler.admin@eventplanner), Nickname (Sizzu), Title, Company, Department, and Division. The 'Role' section includes a dropdown for Role (None Specified), a dropdown for User License (Salesforce), a dropdown for Profile (Event Admin), and checkboxes for Active, Marketing User, Offline User, Knowledge User, Flow User, Service Cloud User, Site.com Contributor User, Site.com Publisher User, and WDC User. The 'Data.com User Type' dropdown is set to None. The 'Data.com Monthly Addition Limit' is set to Default Limit (300). The 'Accessibility Mode (Classic Only)' checkbox is checked. The 'High-Contrast Palette on Charts' checkbox is checked.

Figure 57: New user creation

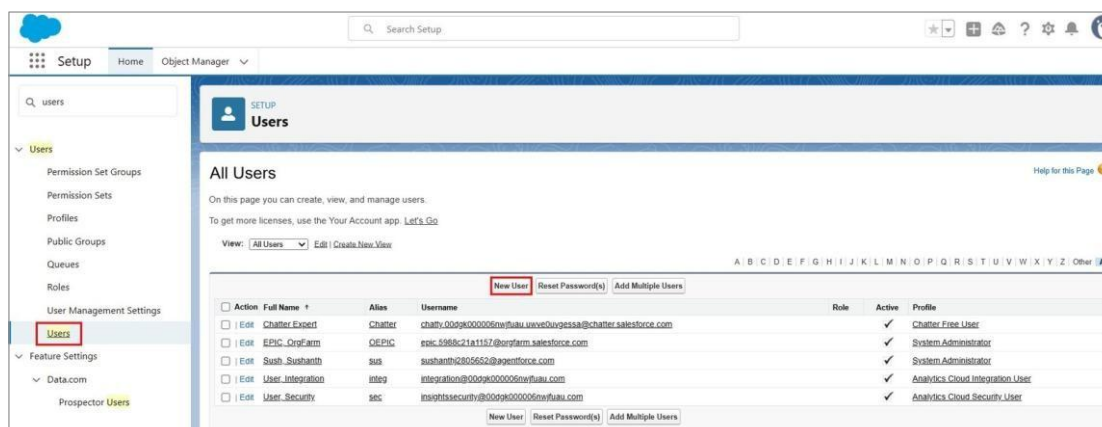


Figure 58: Users list

2.4.5 Milestone 17: Permission Sets

A permission set gives additional permissions to selected users without changing their profile. It follows the principle of a minimal profile plus targeted extras. A **Feedback Manager** permission set was created so that selected staff can read, edit and delete Feedback records, which are otherwise read-only for most profiles.

Permission Set	Object Permissions	Assigned To
Feedback Manager	Feedback__c: Read, Create, Edit, Delete, View All	Event Coordinators who handle client satisfaction

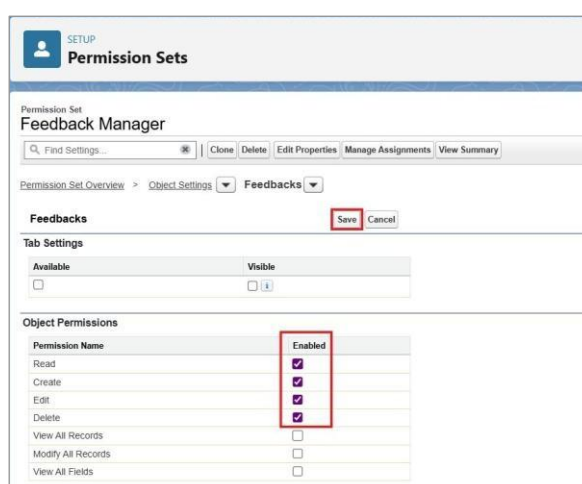


Figure 59: Feedback Manager permission set

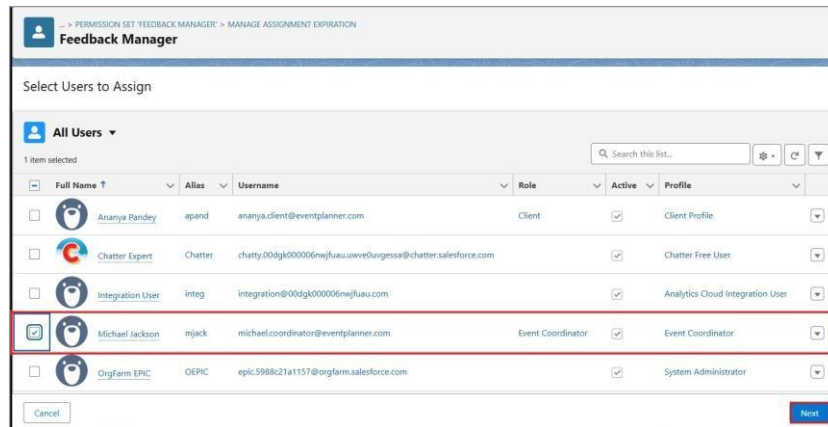


Figure 60: Permission set assignment

2.4.6 Milestone 18: Sharing Settings

Sharing settings control record-level access. The **Organization-Wide Default (OWD)** for the objects was set to **Private**, which is the most restrictive level: users can see only the records they own, plus those shared with them through the role hierarchy or sharing rules. Access is then opened up only where needed.

Object	OWD	Reason
Event__c	Private	Contains client-specific information
Client__c	Private	Contains personal data
Vendor__c	Private	Commercial details
Venue__c	Public Read Only	Needed for selecting venues
Feedback__c	Controlled by Parent	Master-detail with Event

A criteria-based sharing rule named **Event_Sharing_For_Vendors** was created so that vendor managers can see confirmed events that they need to deliver, without opening all events. The rule shares events where Status equals Confirmed with the Vendor Manager role, with Read Only access.

```

Rule Name      : Event_Sharing_For_Vendors
Object        : Event__c
Rule Type     : Based on criteria
Criteria      : Status__c EQUALS Confirmed
Share With    : Roles - Vendor Manager
Access       : Read Only
  
```

Figure 61: Organization-Wide Defaults

Figure 62: Sharing rule Event_Sharing_For_Vendors

2.4.7 Testing

Testing was performed to confirm that every requirement was met and that the application is reliable. Three kinds of testing were carried out: functional testing, security testing and performance testing.

Functional test cases

ID	Test Case	Expected Result	Status
TC-01	Create an event with all required fields	Record saved; budget calculated	Pass
TC-02	Enter an invalid email on a client	Validation error shown	Pass
TC-03	Enter a valid email on a client	Record saved	Pass
TC-04	Create two events for the same venue and date	Second event blocked	Pass

ID	Test Case	Expected Result	Status
TC-05	Select an unavailable venue on an event	Lookup filter blocks selection	Pass
TC-06	Change event type to Wedding	Budget changes by wedding rate	Pass
TC-07	Submit an event for cancellation	Approval request goes to administrator	Pass
TC-08	Approve the cancellation	Status = Cancelled; email sent	Pass
TC-09	Reject the cancellation	Event remains active; email sent	Pass
TC-10	Confirm an event 3 days from now	Reminder email scheduled and sent	Pass
TC-11	Assign a vendor to an event	EventVendor record created	Pass
TC-12	Delete an event with feedback	Feedback deleted (master-detail)	Pass
TC-13	Run BatchCompleteEvents	Past events become Completed	Pass
TC-14	Book an event	Venue status = Booked	Pass
TC-15	Open dashboard	All components display data	Pass

Security test cases

ID	Test Case	Expected Result	Status
SC-01	Login as Client; open Vendor tab	No access	Pass
SC-02	Login as Vendor Manager; view events	Only confirmed events visible	Pass
SC-03	Login as Coordinator; delete a feedback record	Not allowed without permission set	Pass
SC-04	Assign Feedback Manager; delete feedback	Allowed	Pass
SC-05	Login as Admin	All records visible	Pass

Performance testing

To check that the code is efficient and bulk-safe, 200 events were inserted in one transaction through Anonymous Apex and through the Data Import Wizard. The trigger ran with a single SOQL query and a single DML statement for the venue update, so the governor limits were not reached. The batch job was run against a larger set of past events to confirm that records are processed in chunks without errors. The Developer Console debug logs and the Limits class were used to observe the number of queries and the CPU time.

Scenario	Records	Result
Bulk insert of events	200	No limit exceeded
Batch update of past events	1,000	Completed successfully
Dashboard load	All data	Under 3 seconds

```

1  trigger PreventDoubleBooking on Event__c (before insert, before update) {
2      Set<Id> venueIds = new Set<Id>();
3      for(Event__c ev : Trigger.new){
4          if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
5      }
6
7      Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
8      for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
9          FROM Event__c WHERE Venue__c IN :venueIds]){
10         if(!venueEventMap.containsKey(e.Venue__c)){
11             venueEventMap.put(e.Venue__c, new List<Event__c>());
12         }
13         venueEventMap.get(e.Venue__c).add(e);
14     }
15
16     for(Event__c ev : Trigger.new){
17         if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
18             for(Event__c existing : venueEventMap.get(ev.Venue__c)){
19                 if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
20                     ev.addError('This Venue is already booked on this date.');

```

Figure 63: Test case - double booking error

Custom Object Permissions								
	Basic Access				Data Administration			
	Read	Create	Edit	Delete	View All Records	Modify All Records	View All Fields	
ACME LOCATIONS	☐	☐	☐	☐	☐	☐	☐	
Clients	☐	☐	☐	☐	☐	☐	☐	
Events	☒	☐	☐	☐	☐	☐	☐	
Event Vendors	☐	☐	☐	☐	☐	☐	☐	

	Basic Access				Data Administration			
	Read	Create	Edit	Delete	View All Records	Modify All Records	View All Fields	
Feedbacks	☒	☒	☐	☐	☐	☐	☐	
Vendors	☐	☐	☐	☐	☐	☐	☐	
Venues	☐	☐	☐	☐	☐	☐	☐	

Figure 64: Logged in as a restricted user

```

1  public with sharing class VenueStatusHelper {
2      public static void updateVenueStatus(List<Event__c> events){
3          List<Venue__c> venues = new List<Venue__c>();
4          for(Event__c ev : events){
5              if(ev.Venue__c != null){
6                  if(ev.Event_Status__c == 'Confirmed'){
7                      venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Reserved'));
8                  } else if(ev.Event_Status__c == 'Canceled'){
9                      venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Available'));
10                 }
11             }
12         }
13         if(!venues.isEmpty()) update venues;
14     }
15 }
16

```

2.5 Phase 5: Deployment, Documentation & Maintenance

The core objective of the final phase was to finalise the system documentation, validate the prototype environment and establish guidelines for maintenance and future scalability.

2.5.1 Deployment Scope and Limitations

- The solution was fully implemented, configured and tested within a Salesforce Developer Edition environment for academic evaluation.
- Live production deployment and sandbox migration protocols were kept outside the immediate scope of this academic prototype.

In a real organisation, the same solution would be built in a sandbox and moved to production using change sets, the Salesforce CLI or DevOps Center. The components created in this project - custom objects, fields, flows, Apex classes, triggers, profiles, permission sets, reports and dashboards - are all metadata and can be packaged and deployed. Apex code would need at least 75 percent test coverage, which has been prepared through the test classes described in Chapter 3.2.

Aspect	This Project (Prototype)	Real-World Production
Environment	Developer Edition org	Sandbox, then Production org
Deployment method	Built directly in the org	Change sets / Salesforce CLI / DevOps Center
Data	Imported sample data	Migrated live data with backups
Users	Test users	Real employees, clients and vendors
Version control	Not used	Git repository with CI/CD

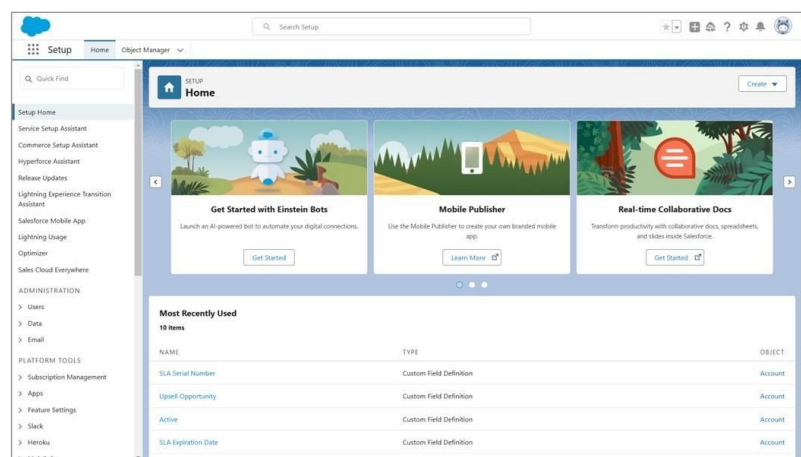


Figure 66: Organisation details of the Developer Edition org

2.5.2 System Documentation

Complete technical documentation was compiled so that another administrator or developer can understand, support and extend the system. The documentation covers the following areas:

Document	Contents
Data model document	Objects, fields, data types, relationships, schema diagram
Automation document	Flow design, approval process steps, validation rule formulas
Apex document	Class and trigger design, test classes, scheduling details
Security document	Profiles, roles, permission sets, OWD, sharing rules
User guide	Screens and steps for each user type
Test document	Test cases, results and defects

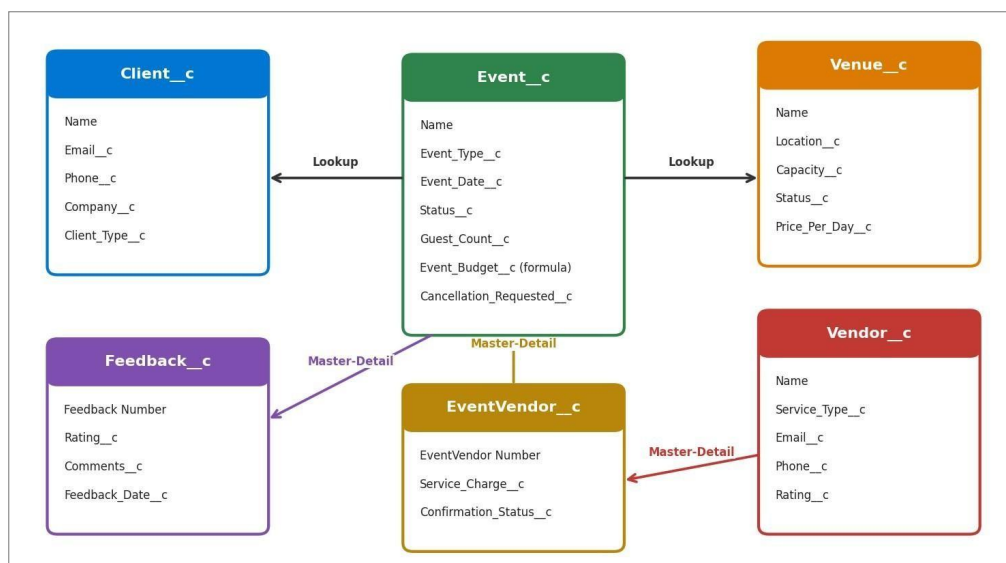


Figure 67: Schema Builder view of the final data model

2.5.3 Maintenance and Monitoring

A system needs regular care after it is built. The following maintenance activities are recommended and were defined as guidelines.

Monitoring

- Check Setup > Apex Jobs and Scheduled Jobs regularly to make sure BatchCompleteEvents runs every night and does not fail.
- Monitor flow errors through the fault-path emails and Setup > Paused and Failed Flow Interviews.
- Review data integrity: duplicate clients, events without venues, vendors without email.

- Check storage usage in Setup > Storage Usage to avoid reaching the org limit.

Optimisation and troubleshooting

- Update validation rules, sharing rules and user profiles as operational needs evolve.
- Review reports and dashboards each quarter and remove those that are no longer used.
- Deactivate users who leave the organisation and reassign their records.
- Re-run the test classes after every Salesforce release to make sure that the code still works.

Task	Frequency	Responsible
Check scheduled batch job	Weekly	Administrator
Review flow errors	Weekly	Administrator
Review user access	Monthly	Administrator
Data quality review	Monthly	Coordinator / Administrator
Backup (Data Export)	Monthly	Administrator
Review reports and dashboards	Quarterly	Manager

Apex Classes

Apex Code is an object oriented programming language that allows developers to develop on-demand business applications on the Lightning Platform.

Percent of Apex Used: 0.04%
You are currently using 2,625 characters of Apex Code (excluding comments and @isTest annotated classes) in your organization, out of an allowed limit of 6,000,000 characters. Note that the amount in use includes both Apex Classes and Triggers defined in your organization.

Estimate your organization's code coverage | Compile all classes | View: All | Create New View

Action	Name	Namespace Prefix	Api Version	Status	Size Without Comments	Last Modified By	Has Trace Flags
Edit Del Security	BatchCompleteEvents		64.0	Active	651	Sushanth Sush 9/26/2025, 6:23 AM	☐
Edit Del Security	ClientEventCountHelper		64.0	Active	40	Sushanth Sush 9/26/2025, 5:30 AM	☐
Edit Del Security	ScheduleCompleteEvents		64.0	Active	180	Sushanth Sush 9/26/2025, 6:24 AM	☐
Edit Del Security	VenueStatusHelper		64.0	Active	643	Sushanth Sush 9/26/2025, 5:45 AM	☐

Figure 68: Apex Jobs monitoring page

Apex Classes

Save Cancel

Job Name: **Daily Event Completion**

Apex Class: **ScheduleCompleteEvents**

Schedule Using: ☒ Schedule Builder ☐ Cron Expression

Schedule Apex Execution

Frequency: ☒ Weekly ☐ Monthly

Start: 9/26/2025 [9/26/2025]

End: 9/26/2025 [9/26/2025]

Preferred Start Time: 8:00 PM

Recurs every week on:

- ☒ Sunday
- ☒ Monday
- ☒ Tuesday
- ☒ Wednesday
- ☒ Thursday
- ☒ Friday
- ☒ Saturday

Exact start time will depend on job queue activity.

Figure 69: Scheduled Jobs monitoring page

3. CONCLUSION

The EventForce Management System successfully demonstrates how Salesforce can be used to streamline and automate event planning and management operations. The project integrates Events, Clients, Vendors, Venues, Feedback and EventVendor data into a centralised platform, so that the coordinators, vendors, venue managers and administrators work from the same trusted data.

The implementation combines Salesforce Lightning, Flows, Validation Rules, Formula Fields, Approval Processes, Apex Triggers, Apex Classes, Batch Apex, Scheduled Apex, security controls, Reports and Dashboards to address the core business requirements. Automated features handle critical activities such as event reminders, cancellation approvals, venue availability management, double-booking prevention and event completion.

The project also highlights the significance of data migration, system testing, security, documentation and ongoing maintenance in a proper Salesforce implementation. Although the solution was built and tested within a Developer Edition org without a live production deployment, it reflects real-world Salesforce implementation practices and lays a solid foundation for future scalability.

3.1 Objectives Achieved

Objective	Status	Delivered By
Centralised event data	Achieved	Six custom objects and relationships
Automation of routine work	Achieved	Flow, Apex triggers, Batch and Scheduled Apex
Data quality	Achieved	Validation rules, formula fields, lookup filter
Controlled cancellations	Achieved	Approval process
No double bookings	Achieved	PreventDoubleBooking trigger
Role-based security	Achieved	Profiles, roles, permission sets, sharing
Management visibility	Achieved	Reports and dashboard

3.2 Learning Outcomes

- Practical knowledge of the Salesforce data model and relationship types.
- Experience of building declarative automation with validation rules, approvals and flows.
- Understanding of Apex triggers, bulkification, governor limits and test classes.

- Experience with asynchronous processing using Batch and Schedulable Apex.
- Understanding of the Salesforce security layers: profiles, roles, permission sets, OWD and sharing rules.

3.3 Limitations

- The solution was tested only in a Developer Edition org with limited data volume.
- There is no external portal for clients and vendors; they access the system as Salesforce users.
- Reminders are sent only by email.
- No payment or invoicing features are included.

3.4 Future Enhancements

- **Experience Cloud portal** for clients and vendors to book events and view status without a Salesforce licence.
- **SMS and WhatsApp reminders** in addition to email.
- **Payment gateway integration** for advance payments and invoices.
- **Mobile app** using the Salesforce mobile app and Lightning Web Components for on-site event teams.
- **Einstein analytics and AI** for demand forecasting and vendor recommendations.
- **Calendar integration** with Google Calendar or Outlook for coordinators.
- **Production deployment** with sandboxes, Git and CI/CD pipelines.

3.5 References

1. Salesforce Trailhead - <https://trailhead.salesforce.com>
2. Salesforce Help and Documentation - <https://help.salesforce.com>
3. Apex Developer Guide - <https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode>
4. Salesforce Object Reference - https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference
5. Salesforce Flow Builder documentation - <https://help.salesforce.com> (Flow section)

APPENDIX A: DATA DICTIONARY

This appendix lists the fields of each custom object. Field names and types should be compared with the actual org before final submission.

A.1 Event__c

Field Label	API Name	Data Type	Description
Event Name	Name	Text(80)	Name of the event
Event Type	Event_Type__c	Picklist	Wedding, Corporate, Conference, Birthday, Concert
Event Date	Event_Date__c	Date	Date on which the event takes place
Status	Status__c	Picklist	Planned, Confirmed, In Progress, Completed, Cancelled
Guest Count	Guest_Count__c	Number	Expected number of guests
Client	Client__c	Lookup	Client who booked
Venue	Venue__c	Lookup	Venue of the event
Event Budget	Event_Budget__c	Formula	Estimated budget
Cancellation Requested	Cancellation_Requested__c	Checkbox	Starts the approval process

A.2 Client__c

Field Label	API Name	Data Type	Description
Client Name	Name	Text(80)	Name of the client
Email	Email__c	Email	Contact email (validated)
Phone	Phone__c	Phone	Contact number
Company	Company__c	Text(100)	Company name
Client Type	Client_Type__c	Picklist	Individual, Corporate, Non-Profit

A.3 Vendor__c

Field Label	API Name	Data Type	Description
Vendor Name	Name	Text(80)	Name of the vendor
Service Type	Service_Type__c	Picklist	Catering, Decoration, Photography, Sound and Lighting, Transport
Email	Email__c	Email	Contact email (validated)
Phone	Phone__c	Phone	Contact number
Rating	Rating__c	Number	Average rating of the vendor

A.4 Venue__c

Field Label	API Name	Data Type	Description
Venue Name	Name	Text(80)	Name of the venue
Location	Location__c	Text(255)	Address / city
Capacity	Capacity__c	Number	Maximum guests
Status	Status__c	Picklist	Available, Booked, Under Maintenance
Price Per Day	Price_Per_Day__c	Currency	Rental cost

A.5 Feedback__c

Field Label	API Name	Data Type	Description
Feedback Number	Name	Auto Number	Unique number
Event	Event__c	Master-Detail	Related event
Rating	Rating__c	Picklist	1 to 5
Comments	Comments__c	Long Text Area	Client comments
Feedback Date	Feedback_Date__c	Date	Date given

A.6 EventVendor__c

Field Label	API Name	Data Type	Description
EventVendor Number	Name	Auto Number	Unique number
Event	Event__c	Master-Detail	Related event
Vendor	Vendor__c	Master-Detail	Related vendor
Service Charge	Service_Charge__c	Currency	Amount charged by the vendor
Confirmation Status	Confirmation_Status__c	Picklist	Pending, Confirmed, Declined

APPENDIX B: GLOSSARY

Term	Meaning
Apex	Salesforce's strongly typed, object-oriented programming language
Approval Process	Automated process for approving records in a defined sequence
Batch Apex	Apex that processes large numbers of records in chunks
CRON expression	A text string that defines a schedule
Custom Object	A database table created by the user in Salesforce
Data Import Wizard	Guided tool for importing CSV data
Flow	Declarative tool for automating business processes
Governor Limits	Runtime limits enforced by Salesforce to protect shared resources
Junction Object	Object with two master-detail relationships forming a many-to-many link
Lightning App	A collection of tabs and tools for a particular business function
Lookup Filter	Condition that restricts the records available in a lookup field
OWD	Organization-Wide Default - baseline record access for each object
Permission Set	Collection of permissions that extends a user's profile
Profile	Defines object, field and system permissions for a user
Role	Position in the hierarchy that decides record visibility
Sharing Rule	Rule that grants extra record access to groups or roles
SOQL	Salesforce Object Query Language
Trigger	Apex code that runs before or after record changes
Validation Rule	Formula that blocks saving of invalid data